

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Docente:
Prof. José Canelón, Ph.D.

UNIDAD II

RED PERCEPTRÓN MULTICAPA

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Contenido

- Redes neuronales shallow
- Funciones de activación
- Red perceptrón multicapa (MLP)
- Retropropagación y regla de la cadena
- Regularización
- Optimizadores. Learning rate scheduling. Inicialización de pesos
- Debugging: gradient flow, vanishing/exploding gradients, sanity checks.
- Monitoreo de experimentos con Weights & Biases

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Neuronales

- Una red neuronal es un sistema de procesamiento paralelo, masivo y distribuido, formado por unidades simples que tienen la capacidad de adquirir conocimiento a través de un proceso de entrenamiento, y almacenar este conocimiento para su uso posterior.
- Existen diferentes tipos de redes neuronales, entre los que pueden mencionarse
 - Redes perceptrón multicapa
 - Redes convolucionales
 - Redes recurrentes
 - Transformers

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa

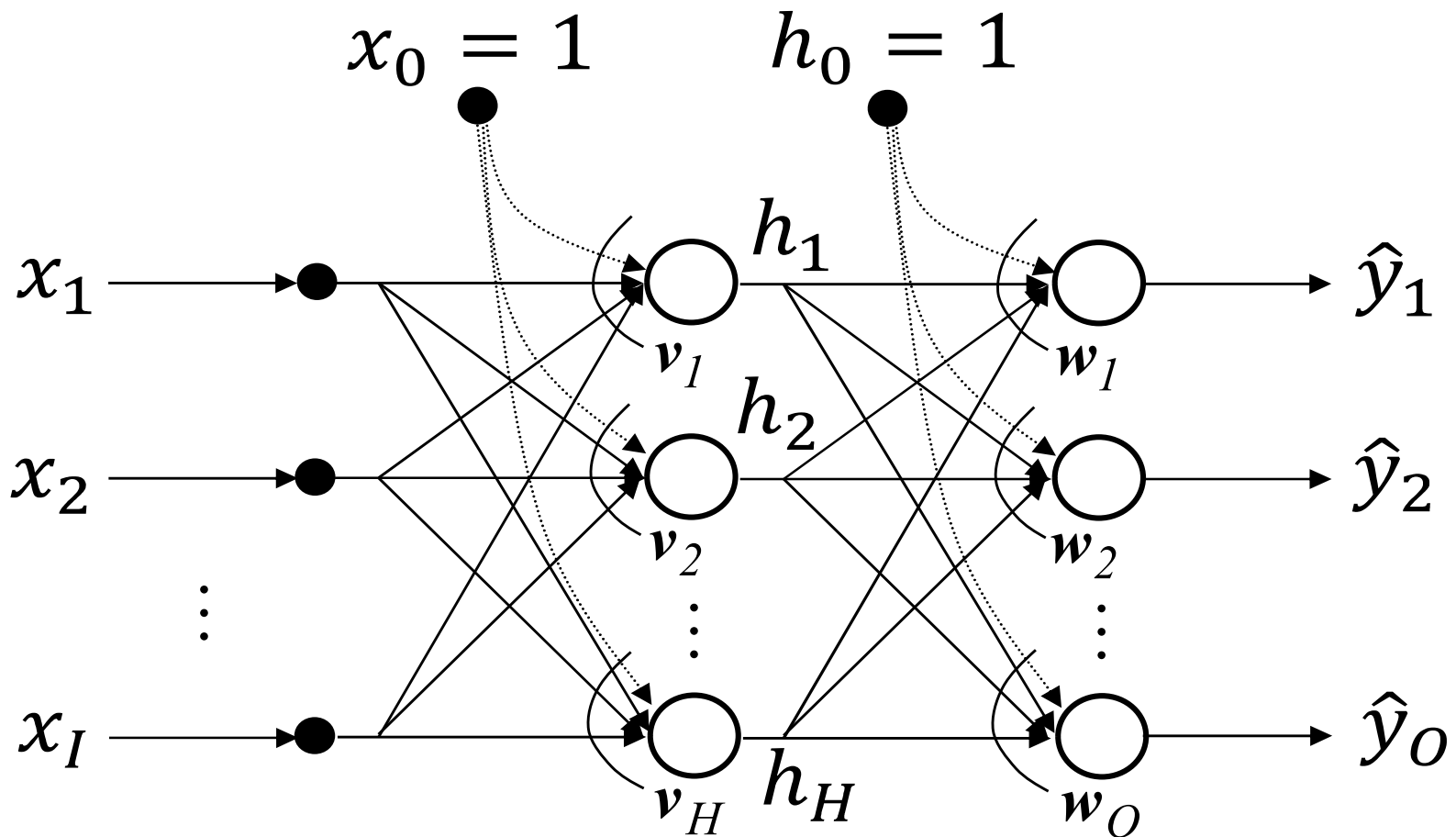
- También se conocen como Redes Feedforward o Redes Densas
- Están constituidas por varias capas de perceptrones (nodos), conectadas en forma secuencial.
- Dos capas consecutivas están completamente conectadas
- Existen tres tipos de capas:
 1. capa de entrada, en la que se aplica el vector de entrada de la red.
 2. capa de salida, en la que aparece el vector de salida que produce la red.
 3. capas ocultas, que se ubican entre la capa de entrada y la capa de salida. Puede haber una o más capas ocultas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa

Una red shallow tiene una capa oculta entre la entrada y la salida



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa

Pesos entre la capa de entrada y la capa oculta

$$\mathbf{V} = \begin{bmatrix} v_{10} & v_{11} & v_{12} & \cdots & v_{1I} \\ v_{20} & v_{21} & v_{22} & \cdots & v_{2I} \\ \vdots & \vdots & \vdots & & \vdots \\ v_{H0} & v_{H1} & v_{H2} & \cdots & v_{HI} \end{bmatrix} \in \mathbb{R}^{H \times (I+1)}$$

Pesos entre la capa oculta y la capa de salida

$$\mathbf{W} = \begin{bmatrix} w_{10} & w_{11} & w_{12} & \cdots & w_{1H} \\ w_{20} & w_{21} & w_{22} & \cdots & w_{2H} \\ \vdots & \vdots & \vdots & & \vdots \\ w_{O0} & w_{O1} & w_{O2} & \cdots & w_{OH} \end{bmatrix} \in \mathbb{R}^{O \times (H+1)}$$

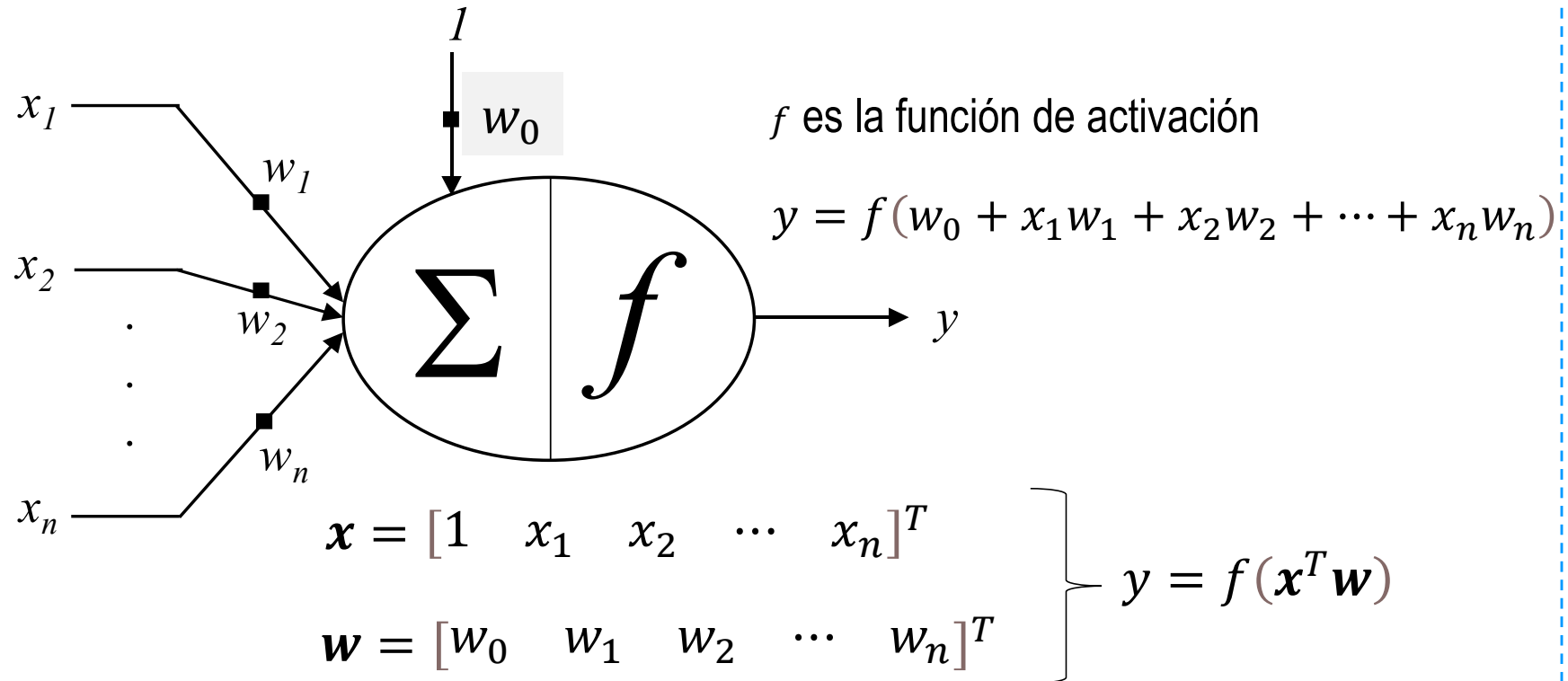
Número total de pesos: $H \times (I + 1) + O \times (H + 1)$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa El Perceptrón

- Unidad computacional básica inspirada en la neurona biológica (McCulloch & Pitts, 1943)



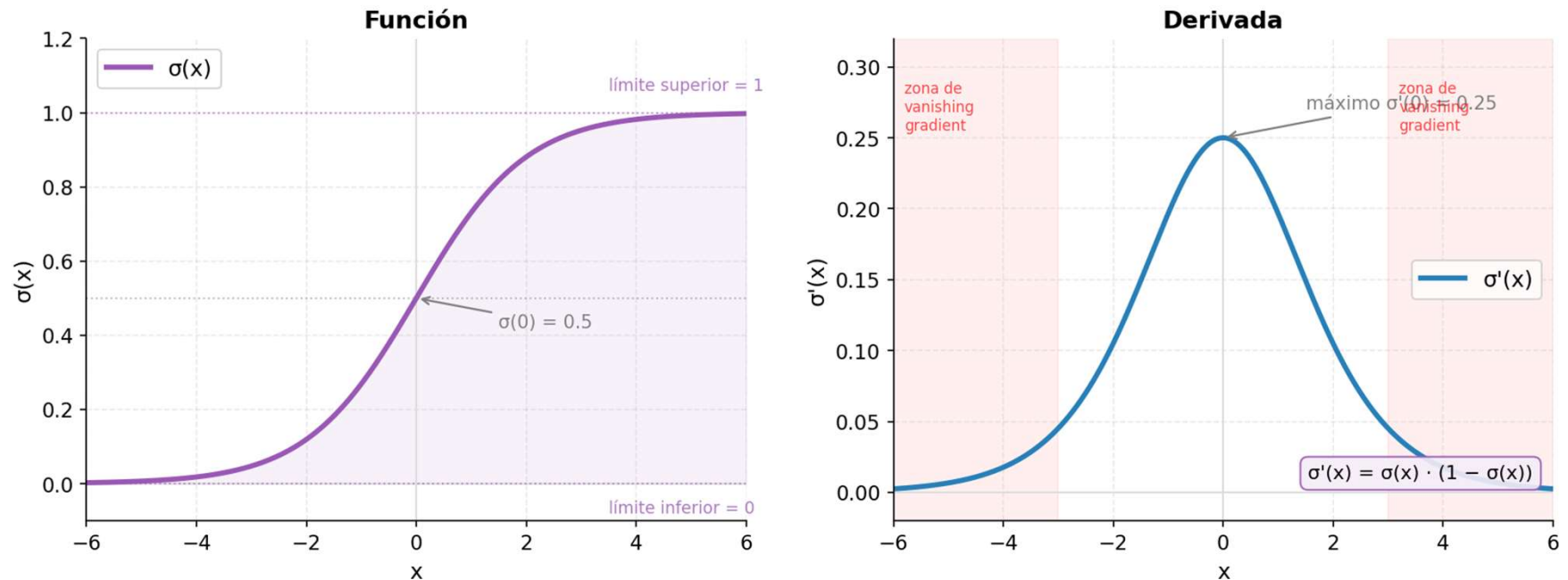
Redes Perceptrón Multicapa **El Perceptrón**

- La función de activación $f(\cdot)$ debe ser no lineal
 - Si $f(\cdot)$ no es no lineal: la red completa colapsa a una transformación lineal
 - Si $f(\cdot)$ es no lineal: la red puede representar funciones arbitrariamente complejas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Funciones de Activación Sigmoide

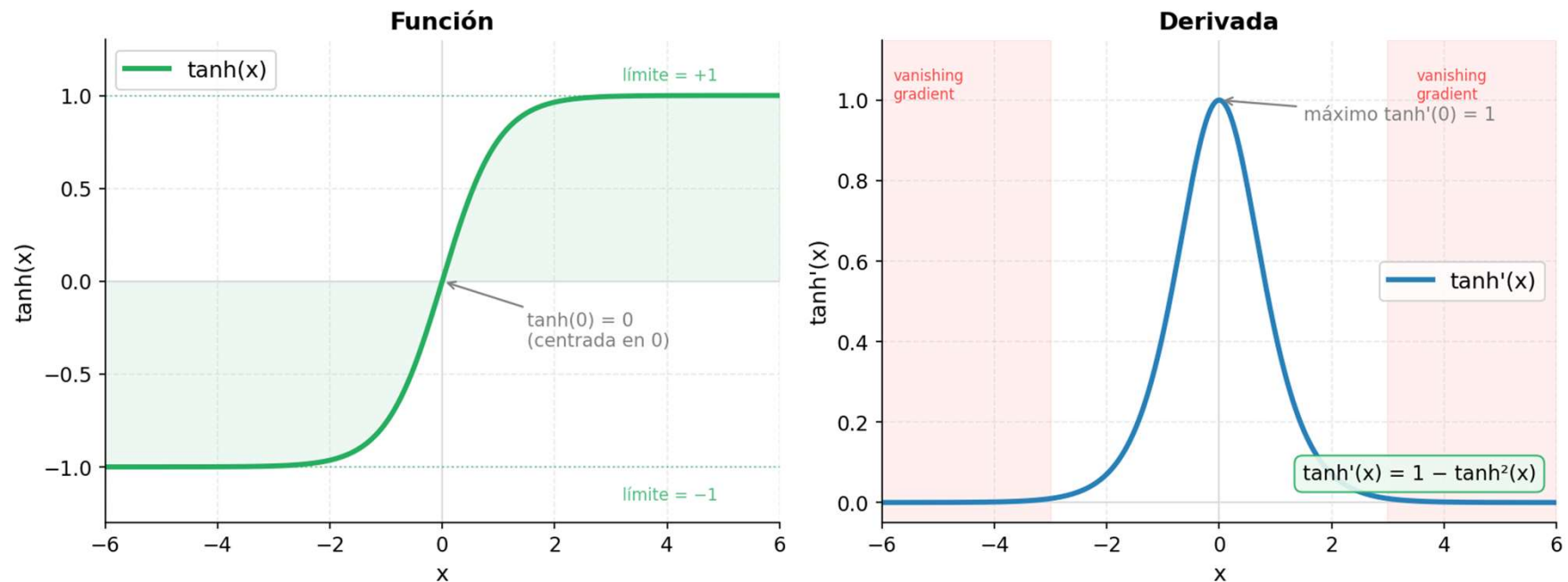


$$f(x) = \frac{1}{1 + e^{-x}} \Rightarrow f'(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Funciones de Activación Tangente hiperbólica

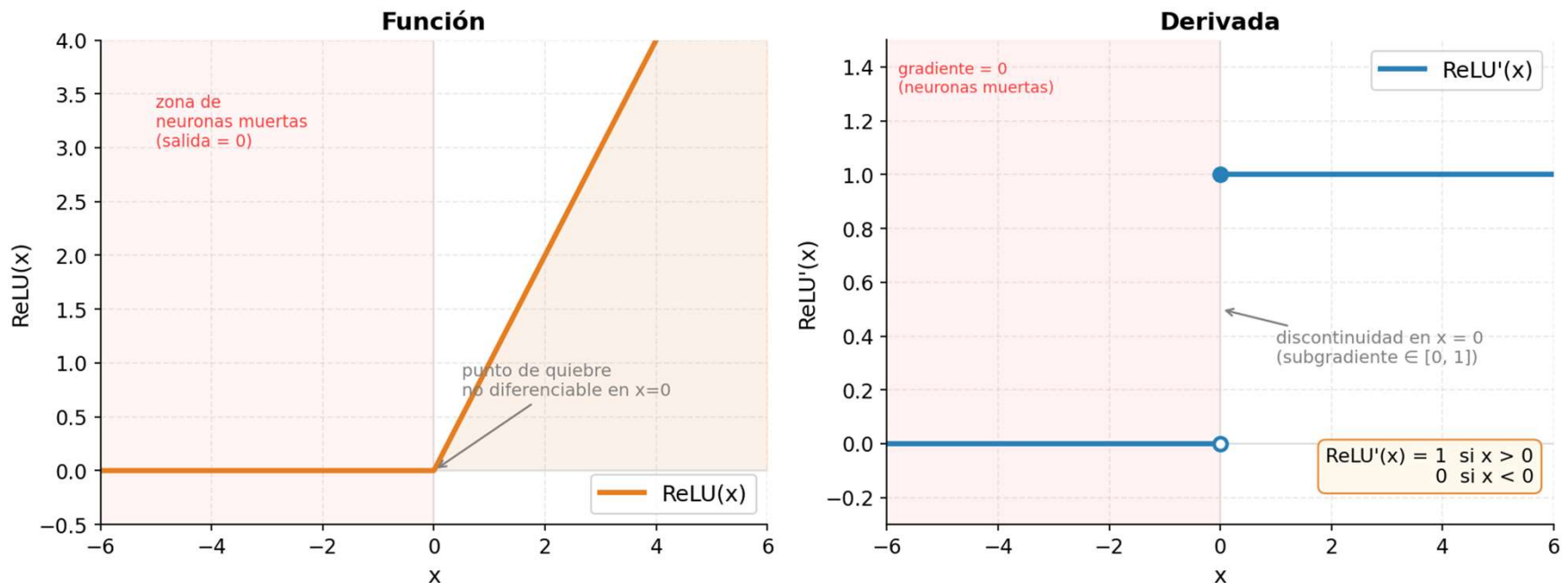


$$f(x) = \tanh(x) \Rightarrow f'(x) = \text{sech}^2 x$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Funciones de Activación Rectified Linear Unit (ReLU)

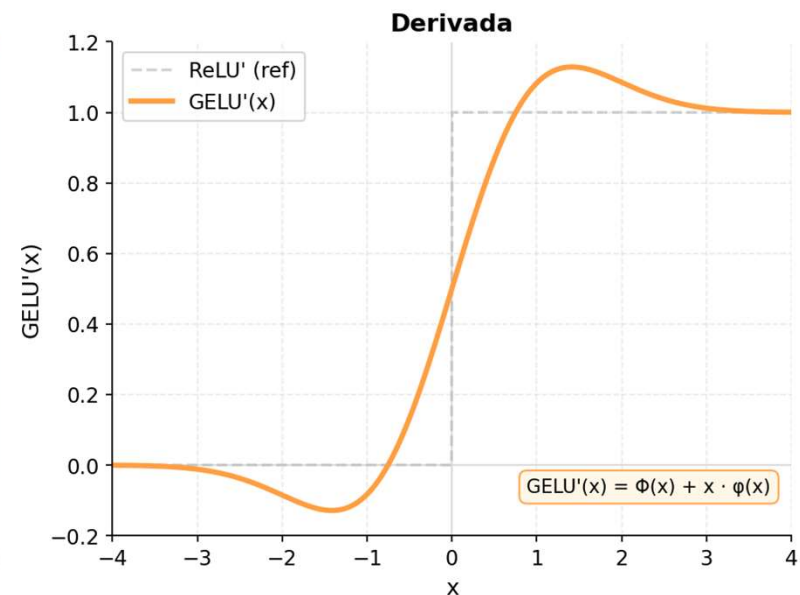
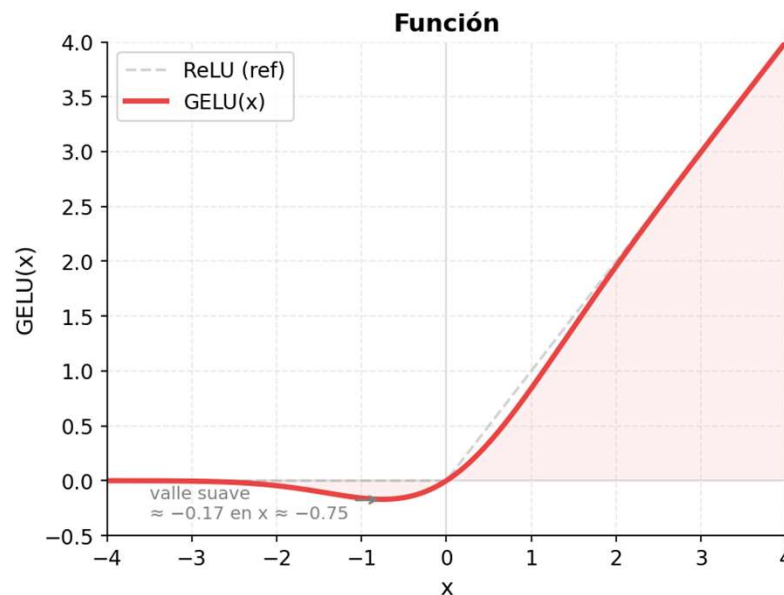


$$f(x) = \begin{cases} 0 & \text{para } x < 0 \\ x & \text{para } x > 0 \end{cases} \Rightarrow f'(x) = \begin{cases} 0 & \text{para } x < 0 \\ 1 & \text{para } x > 0 \end{cases}$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Funciones de Activación Gaussian Error Linear Unit (GELU)



Ecuación
Exacta

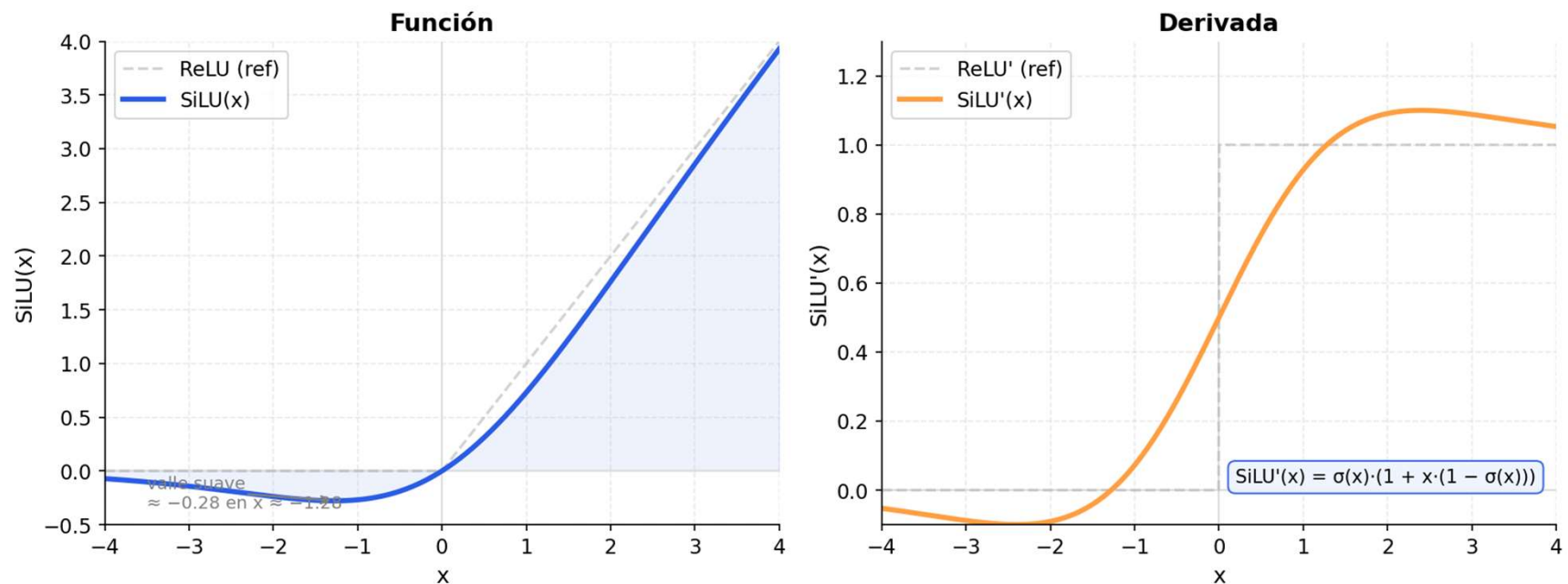
$$GELU(x) = x \cdot \Phi(x)$$

Ecuación
Aproximada
(PyTorch)

$$GELU(x) \approx \frac{x}{2} \left[1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715 x^3) \right) \right]$$

$$GELU'(x) = \Phi(x) + x \phi(x)$$

Redes Perceptrón Multicapa Funciones de Activación Sigmoid Linear Unit (SiLU/Swish)



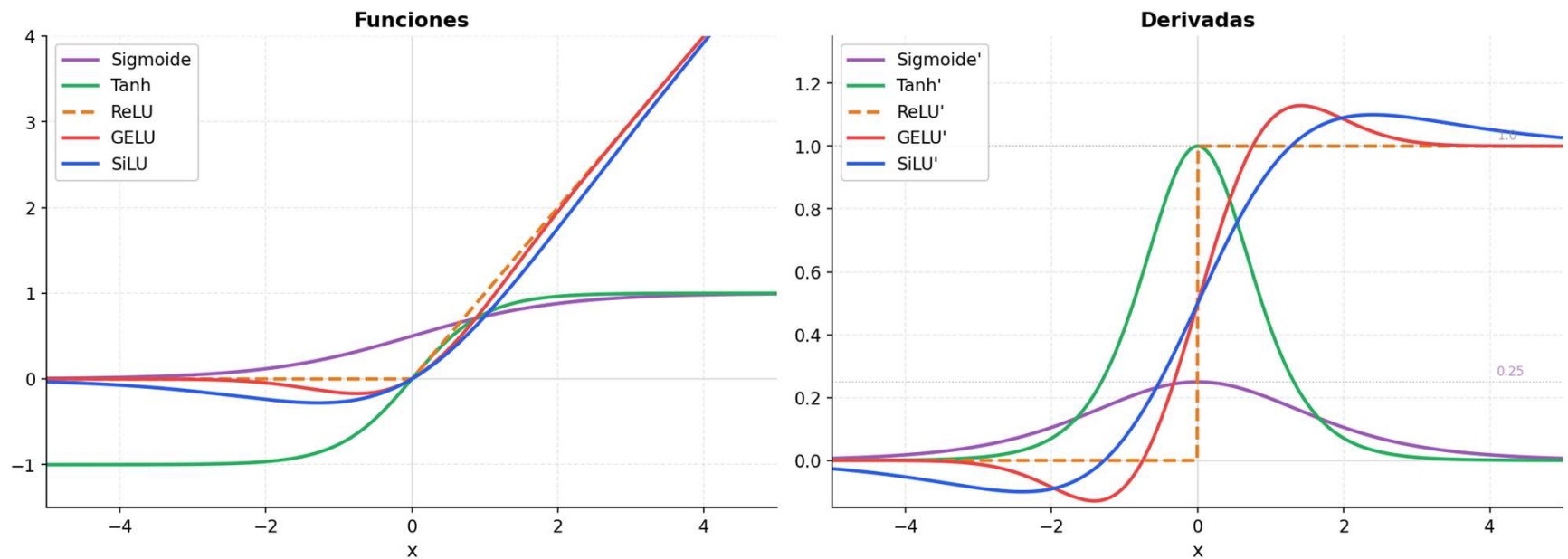
$$SiLU(x) = \frac{x}{1 + e^{-x}} \Rightarrow SiLU'(x) = SiLU(x) + \sigma(x)(1 - SiLU(x))$$

donde $\sigma(x)$ es la sigmoide

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Comparativa de Funciones de Activación



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Comparativa de Funciones de Activación

	ReLU	GELU	SiLU
Diferenciable en 0	No	Sí	Sí
Valores negativos	Exactamente 0	≈ 0 (suave)	≈ 0 (suave)
Costo computacional	Muy bajo	Medio	Bajo
Uso principal	CNNs clásicas RPMC	Transformers (NLP/ViT)	CNNs modernas, YOLO

La ventaja clave de SiLU y GeLU sobre ReLU es que son **suaves y no monótonas**: tienen un pequeño "valle" para valores levemente negativos (~ -0.1 para $x \approx 0.5$). Esto empíricamente mejora la capacidad expresiva de la red y el flujo de gradientes durante el entrenamiento.

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Elección de la Función de Activación

- La elección de la función de activación ha sido uno de los factores más determinantes en la evolución del Deep Learning

Cronología

- 1943 — McCulloch & Pitts: neurona binaria (función escalón)
- 1958 — Rosenblatt: Perceptrón con función escalón
- 1986 — Rumelhart et al.: backpropagation con Sigmoides (diferenciable)
- 1990s-2000s — *tanh* reemplaza a Sigmoides en capas ocultas (Lecun et al., 1998)
- 2010 — Nair & Hinton: *ReLU* muestra clara ventaja sobre *tanh* en CNNs
- 2012 — AlexNet: *ReLU* + Dropout → revolución en ImageNet
- 2016 — Clevert et al.: ELU; He et al.: Leaky *ReLU* con análisis
- 2016 — Hendrycks & Gimpel: GELU — adoptada por BERT, GPT y todos los Transformers
- 2017 — Ramachandran et al.: Swish (SiLU) via búsqueda automática

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Problemas de la Sigmoide en Redes Profundas

Problema 1: Vanishing gradient

- Para $|x| > 3$, el gradiente prácticamente desaparece
- En backpropagation: el gradiente se multiplica en cada capa
- Con L capas: gradiente $\approx (0.25)^L \rightarrow$ exponencialmente pequeño
- Con 10 capas: gradiente $\approx (0.25)^{10} \approx 10^{-6}$ — las capas iniciales no aprenden

Problema 2: Salida no centrada en 0

- $\sigma(x) \in (0, 1)$ siempre positivo
- Gradientes de los pesos de la capa siguiente son siempre del mismo signo
- Provoca actualizaciones 'en zigzag' en el espacio de parámetros

Problema 3: Costo computacional

- e^x es más costoso que $\max(0, x)$ de *ReLU*

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Problemas de la Sigmoide en Redes Profundas

¿Cuándo aún se usa?

- Capa de salida para clasificación binaria
- Gates en LSTMs (con $\tanh$ para cell state)
- Attention weights (junto con softmax)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Función de Activación tanh

Ventaja sobre Sigmoides

- Output centrado en 0 $\rightarrow$ gradientes sin sesgo de signo
- Derivada mayor (hasta 1.0 vs 0.25) $\rightarrow$ entrenamiento más rápido

Mismo problema: vanishing gradient

- Para $|x| > 2$, $\tanh'(x) \approx 0$ — sigue saturando
- Menos grave que sigmoides pero presente en redes muy profundas

Uso actual

- Cell state y output gate en LSTMs y GRUs (Unidad VI del curso)
- Redes shallow para tareas con salida en $(-1, 1)$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Vanishing Gradients

Consecuencia práctica

- Las primeras capas reciben gradientes casi nulos
- Sus pesos apenas se actualizan — no aprenden representaciones útiles
- El entrenamiento efectivo se limita a las últimas capas

Soluciones modernas

- ReLU y variantes — no satura para $x > 0$
- Residual connections (ResNet) — bypass que preserva el gradiente
- Batch Normalization — mantiene activaciones en zona activa

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

¿Por qué ReLU funciona mejor que Sigmoid/Tanh?

- No satura para $x > 0$: la derivada es exactamente 1, no se encoge
- Sparse activation: con datasets típicos, ~50% de neuronas están inactivas
 - Con sparsidad, cada sample activa un subconjunto distinto → los conceptos están más separados en el espacio de activaciones
- Eficiencia computacional: $\max(0, x)$ es trivial en GPU

Referencia histórica

- Popularizada por Nair & Hinton (2010); consolidada con AlexNet (Krizhevsky, 2012)
- 'Using ReLU instead of sigmoid/tanh makes training much faster' — Krizhevsky et al

ReLU — El Problema de las Neuronas Muertas

¿Qué es una neurona muerta?

- Una neurona ReLU queda 'muerta' cuando su preactivación es siempre negativa
- Condición: $w^T x + b < 0$ para todas las muestras de entrenamiento
- Consecuencia: salida = 0, gradiente = 0 $\rightarrow$ los pesos no se actualizan nunca más

Causas comunes

- Learning rate muy alto: actualización excesiva que lleva b a valores muy negativos
- Inicialización incorrecta de pesos
- Datos con alta varianza sin normalización previa

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

ReLU — El Problema de las Neuronas Muertas

Impacto práctico

- En redes bien entrenadas: 10-50% de neuronas pueden estar inactivas (normal y esperado)
- Si >80% están muertas: capacity collapse — la red no puede representar bien los datos

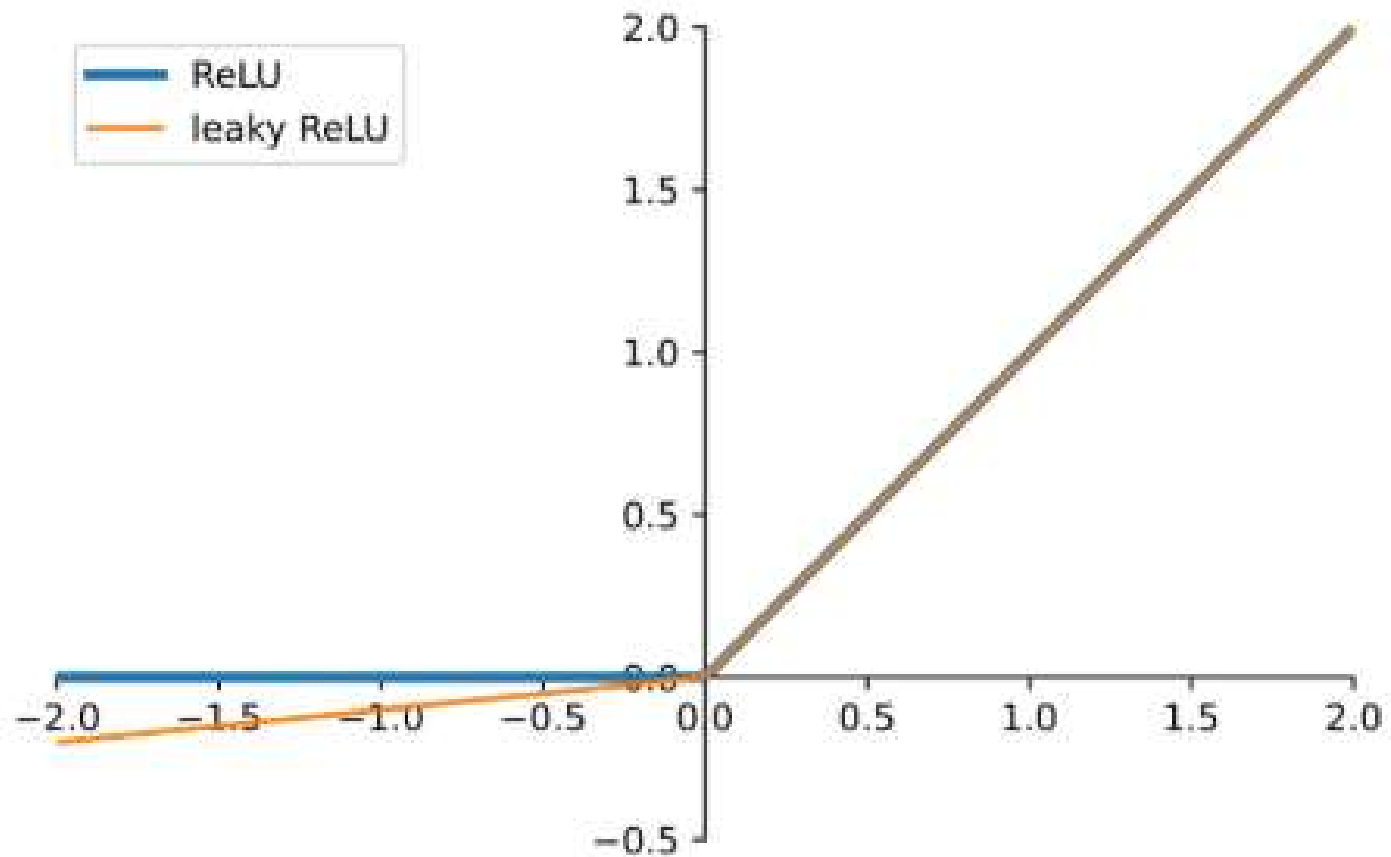
Soluciones

- Leaky ReLU: $f(x) = \max(\alpha x, x)$ con $\alpha = 0.01$ — pequeño gradiente para $x < 0$
- ELU: $f(x) = x$ si $x \geq 0$, $\alpha(e^x - 1)$ si $x < 0$ — suave y continua
- Inicialización cuidadosa (He initialization)
- Batch Normalization: mantiene activaciones en rango activo

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Leaky ReLU

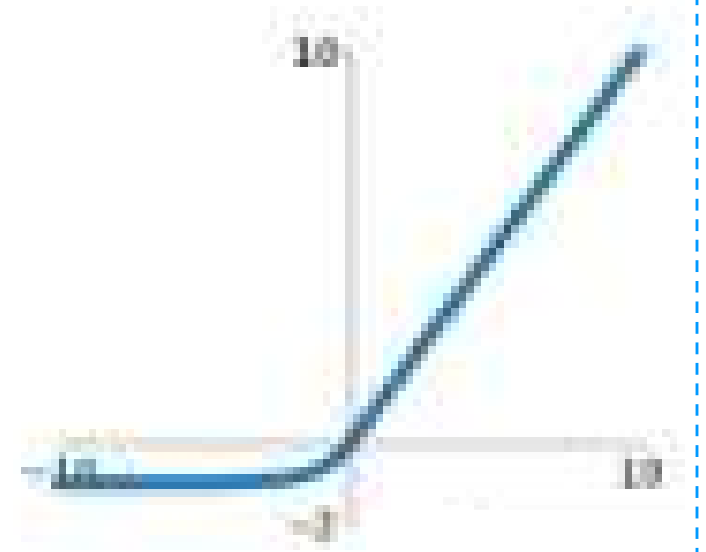


INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

¿Por qué GELU y SiLU Superan a ReLU?

1. Suavidad y diferenciabilidad

- ReLU no es diferenciable en $x=0$ — puede crear inestabilidades en optimización
- GELU y SiLU son C^∞ — gradientes suaves en toda la recta real

2. No monotonía — mayor capacidad expresiva

- El pequeño valle negativo añade curvatura — la red puede representar más funciones con menos neuronas
- Análogo a agregar una pequeña no linealidad extra sin costo de parámetros

3. Gradiente siempre no nulo

- No hay zona donde el gradiente sea exactamente 0 (excepto $x \rightarrow -\infty$)
- Elimina completamente el problema de neuronas muertas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

¿Por qué GELU y SiLU Superan a ReLU?

4. Derivada puede superar 1

- Para $x \approx 1.5$: $\text{GELU}'(x) \approx \text{SiLU}'(x) \approx 1.1$
- Facilita el flujo de gradientes en redes muy profundas

Evidencia empírica

- En Transformers: GELU mejora consistentemente sobre ReLU
- En CNNs de visión: SiLU gana +0.3-0.5% accuracy sobre ReLU

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Saturación

¿Qué es la saturación?

- Una función se satura cuando su derivada tiende a 0 para $|x| \rightarrow \infty$
- La saturación frena el aprendizaje: los gradientes desaparecen

Funciones que saturan

- Sigmoides: $\sigma'(x) \rightarrow 0$ para $|x| > 3$ (satura en ambos extremos)
- Tanh: $\tanh'(x) \rightarrow 0$ para $|x| > 2$ (satura en ambos extremos)

Funciones que no saturan (o saturan solo por un lado)

- ReLU: no satura para $x > 0$, pero tiene zona muerta para $x < 0$
- Leaky ReLU / ELU: no saturan en ningún extremo
- GELU / SiLU: no saturan superiormente ($\rightarrow x$ para $x \gg 0$)
- GELU / SiLU: sí saturan inferiormente ($\rightarrow 0$ para $x \rightarrow -\infty$)

Implicación práctica

- Sigmoides / Tanh: requieren inicialización y normalización cuidadosa para redes profundas
- ReLU y variantes: entrenamiento más estable en redes profundas (> 5 capas)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Flujo de Gradientes

Gradiente durante backpropagation

- El producto de matrices puede explotar o desvanecerse exponencialmente

Derivadas en $x=0$ (punto de operación típico con BN)

- Sigmoides: $\sigma'(0) = 0.25$ ← pequeño, contribuye a vanishing
- Tanh: $\tanh'(0) = 1.0$ ← neutro, mejor que sigmoide
- ReLU: $\text{ReLU}'(0) = 0$ o 1 ← riesgo de neuronas muertas
- GELU: $\text{GELU}'(0) = 0.5$ ← suave, gradiente moderado
- SiLU: $\text{SiLU}'(0) = 0.5$ ← igual que GELU en 0

Gradiente máximo

- Sigmoides: 0.25 — vanishing inevitable en redes profundas
- Tanh: 1.0 — neutro en 0, pero satura rápido
- ReLU: 1.0 para $x > 0$ — sin vanishing en zona activa
- GELU / SiLU: ~ 1.1 para $x \approx 1.5$ — leve amplificación, buena para flujo

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Simetría y Centrado en 0

¿Por qué importa que la activación esté centrada en 0?

- Si $f(x) \geq 0$ siempre (Sigmoides, ReLU), los gradientes tienen todos el mismo signo
- Esto hace que las actualizaciones de los pesos de la capa siguiente sean todas del mismo signo
- Provoca movimiento en zigzag durante la optimización (Lecun et al., 1998)

Análisis por función

- Sigmoides: salida $\in (0,1)$ — no centrada, bias de gradiente
- Tanh: salida $\in (-1,1)$ — centrada en 0, mejor que sigmoides
- ReLU: salida $\in [0,\infty)$ — no centrada, media > 0
- Leaky ReLU/ELU: media ≈ 0 con inicialización correcta
- GELU / SiLU: salida puede ser ligeramente negativa — semicentrada

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Simetría y Centrado en 0

Solución práctica

- Batch Normalization normaliza activaciones a media ≈ 0 después de cada capa
- Elimina la necesidad de preocuparse por el centrado de la activación
- Razón por la que ReLU funciona bien pese a no estar centrada

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Costo Computacional

Costo relativo (operaciones por elemento)

- ReLU: $\max(0, x)$ → 1 comparación — costo mínimo
- Leaky ReLU: $\max(\alpha x, x)$ → 1 comparación + 1 multiplicación
- Sigmoid: $1/(1+\exp(-x))$ → 1 exp + operaciones básicas
- Tanh: $(\exp(x)-\exp(-x))/(\dots)$ → 2 exp + operaciones básicas
- ELU: $\exp(x)-1$ para $x < 0$ → 1 exp (solo para $x < 0$)
- SiLU: $x \cdot \sigma(x)$ → 1 exp + multiplicación
- GELU (exact): $x \cdot \text{erf}(x/\sqrt{2})$ → erf costoso
- GELU (approx): $x \cdot \tanh(\dots)$ → 2 exp + multiplicaciones

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Costo Computacional

En la práctica

- La diferencia es despreciable comparada con las multiplicaciones matriciales dominantes
- ReLU sigue siendo preferida cuando el cómputo es muy limitado
- GELU/SiLU preferidas cuando la precisión importa más (NLP, visión de alta capacidad)

Aceleración hardware

- GPUs modernas tienen instrucciones especializadas para ReLU y sigmoide
- La diferencia real en benchmarks es $< 5\%$ del tiempo de forward pass

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

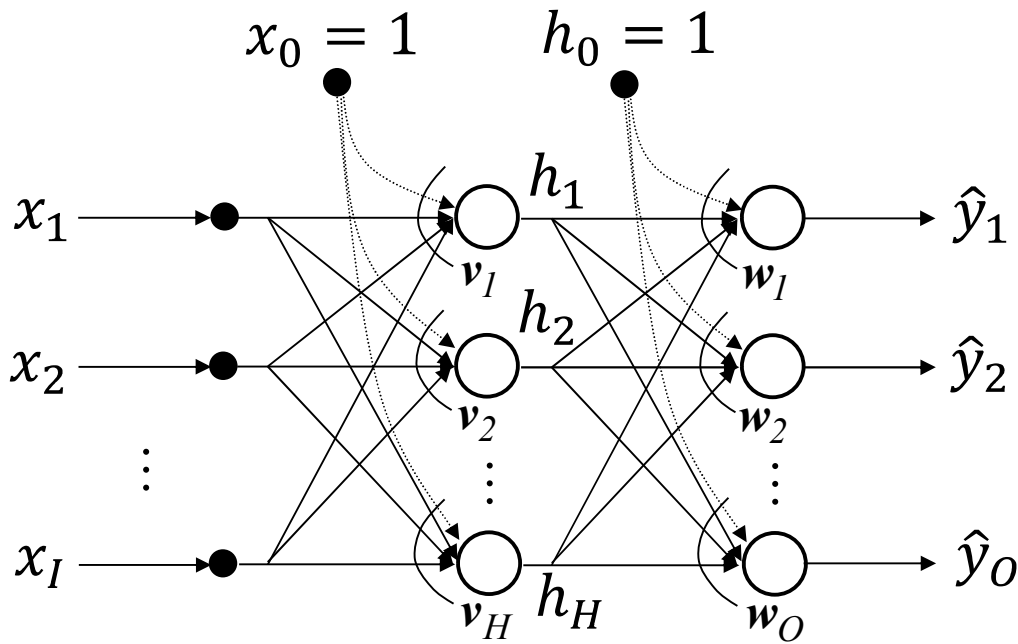
Tabla Comparativa — Funciones de Activación

Función	Rango	Satura	Centrada	Grad. máx.	Uso principal
Sigmoide	$(0,1)$	Ambos lados	No	0.25	Salida binaria, LSTM gates
Tanh	$(-1,1)$	Ambos lados	Sí	1.0	LSTM, GRU, redes shallow
ReLU	$[0,\infty)$	$x < 0$ muerto	No	1.0 ($x > 0$)	CNNs, MLPs clásicos
Leaky ReLU	$(-\infty,\infty)$	No	$\approx$ Sí	1.0	Alternativa a ReLU
ELU	$(-\alpha,\infty)$	$x \rightarrow -\infty$	$\approx$ Sí	1.0	Redes medianas, tabular
GELU	$\approx(-0.17,\infty)$	$x \rightarrow -\infty$	$\approx$ Sí	~ 1.1	Transformers: BERT, GPT
SiLU/Swish	$\approx(-0.28,\infty)$	$x \rightarrow -\infty$	$\approx$ Sí	~ 1.1	YOLO, EfficientNet

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa



$$\mathbf{x} = [x_1 \quad x_2 \quad x_3 \quad \cdots \quad x_I]^T$$

$$\mathbf{x}' = [1 \quad x_1 \quad x_2 \quad \cdots \quad x_I]^T$$

$$\mathbf{h} = f_h(\mathbf{V}\mathbf{x}')$$

$$\mathbf{h} = [h_1 \quad h_2 \quad \cdots \quad h_H]^T$$

$$\mathbf{h}' = [1 \quad h_1 \quad h_2 \quad \cdots \quad h_H]^T$$

$$\hat{\mathbf{y}} = f_y(\mathbf{W}\mathbf{h}')$$

$$\hat{\mathbf{y}} = [\hat{y}_1 \quad \hat{y}_2 \quad \hat{y}_3 \quad \cdots \quad \hat{y}_O]^T$$

A partir de estas ecuaciones, la función que implementa la red puede expresarse (en forma compacta) como $\hat{\mathbf{y}} = \mathcal{F}(\mathbf{x}, \Theta)$ donde Θ es un vector que contiene todos los pesos de la red

Redes Perceptrón Multicapa Teorema de Aproximación Universal

- **Enunciado (Cybenko, 1989 · Hornik, 1991)**

Una red neuronal shallow con suficientes neuronas ocultas y una función de activación no lineal continua puede aproximar cualquier función continua

$$f: \mathbb{R}^D \rightarrow \mathbb{R}$$

con precisión arbitraria sobre un dominio compacto

Condición sobre la función de activación

- Cybenko (1989): demostró para activaciones sigmoidales
- Hornik (1991): generalizó a cualquier activación no constante, acotada y continua
- Leshno et al. (1993): condición mínima = f no polinomial

Redes Perceptrón Multicapa **Teorema de Aproximación Universal**

- **Enunciado (Cybenko, 1989 · Hornik, 1991)**

Implicación práctica

- En principio, una sola capa oculta es suficiente para representar cualquier función
- El problema no es qué puede representar, sino cuántas neuronas necesita
- Para algunas funciones necesita exponencialmente más neuronas que una red profunda

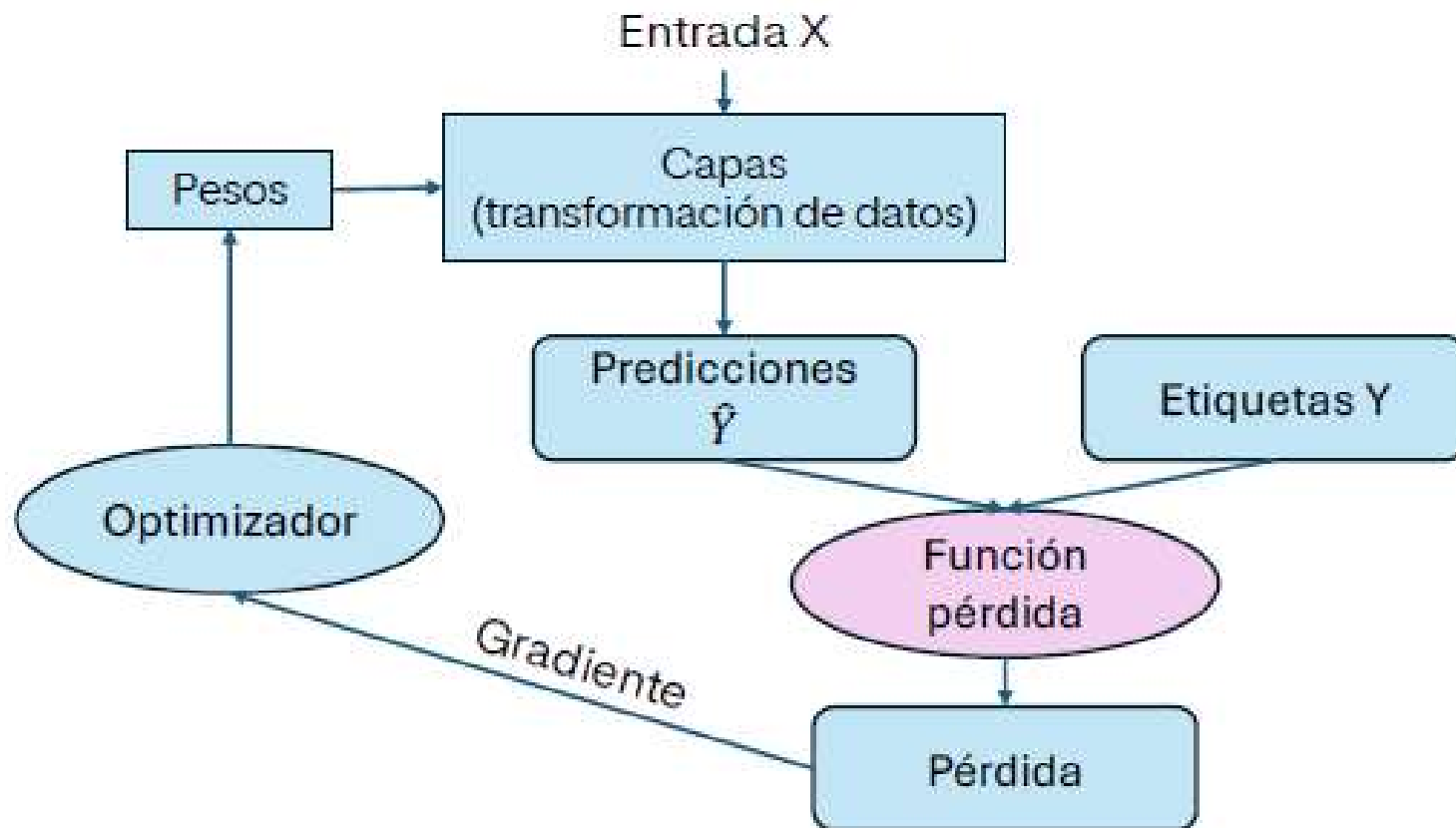
Redes Perceptrón Multicapa Entrenamiento

- El entrenamiento busca encontrar los pesos y sesgos de todas las capas de la red, de modo que la red asigne correctamente las entradas de ejemplo a sus targets (labels-etiquetas) asociados
- La función de pérdida cuantifica la discrepancia entre las predicciones del modelo y los valores reales (ground truth)
 - Función que guía el entrenamiento

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Entrenamiento



Redes Perceptrón Multicapa

Funciones de Pérdida comunes

- Consideremos una red con I entradas y O salidas
- Un dataset de N datos para entrenar esta red tiene la forma

Entrada	Salida
$x_1^{(1)}, x_2^{(1)}, \dots, x_I^{(1)}$	$y_1^{(1)}, y_2^{(1)}, \dots, y_O^{(1)}$
$x_1^{(2)}, x_2^{(2)}, \dots, x_I^{(2)}$	$y_1^{(2)}, y_2^{(2)}, \dots, y_O^{(2)}$
$\vdots$	$\vdots$
$x_1^{(N)}, x_2^{(N)}, \dots, x_I^{(N)}$	$y_1^{(N)}, y_2^{(N)}, \dots, y_O^{(N)}$

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Clasificación

- **Binaria: Binary cross-entropy**

$$\begin{aligned} J(\Theta) &= -\frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \left[y_o^{(n)} \cdot \log(\hat{y}_o^{(n)}) + (1 - y_o^{(n)}) \cdot \log(1 - \hat{y}_o^{(n)}) \right] = \\ &= -\frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \left[y_o^{(n)} \cdot \log(f_o(\mathbf{x}^{(n)}, \Theta)) + (1 - y_o^{(n)}) \cdot \log(1 - f_o(\mathbf{x}^{(n)}, \Theta)) \right] \end{aligned}$$

- **Multiclase: Categorical cross-entropy**

$$J(\Theta) = -\frac{1}{N} \sum_{n=1}^N y_o^{(n)} \cdot \log(\hat{y}_o^{(n)}) = -\frac{1}{N} \sum_{n=1}^N y_o^{(n)} \cdot \log(f_o(\mathbf{x}^{(n)}, \Theta))$$

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Clasificación

Binary cross-entropy

- **Penaliza las predicciones seguras pero incorrectas:** gracias a los términos logarítmicos, la pérdida crece mucho si el modelo asigna probabilidad muy baja a la clase correcta
- **Continua y diferenciable,** lo cual facilita su optimización mediante métodos basados en gradiente
- **Sensible al desbalance entre clases:** se puede usar una variante ponderada
- **Interpretación probabilística:** equivale al negativo del logaritmo de la probabilidad asignada a la clase correcta
- **Se usa en clasificación binaria y conjunta con función sigmoide**

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Clasificación

Categorical cross-entropy

- **Penaliza fuertemente** cuando el modelo asigna baja probabilidad a la clase correcta
- **Continua y diferenciable**, adecuada para optimización de gradiente
- **Enfoque probabilístico**: La salida representa una distribución de probabilidad sobre las clases, es decir

$$\sum_{o=1}^C \hat{p}_{o,n} = 1$$

- **Requisito de one-hot encoding**: asume que las etiquetas objetivo son vectores one-hot
- **Función de pérdida estándar** en clasificación multiclase con salida softmax

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Regresión

- **Mean squared error (MSE) (Pérdida L2)**

$$J(\Theta) = \frac{1}{2} \sum_{n=1}^N \sum_{o=1}^O \left(y_o^{(n)} - \hat{y}_o^{(n)} \right)^2 = \frac{1}{2} \sum_{n=1}^N \sum_{o=1}^O \left(y_o^{(n)} - f_o(\mathbf{x}^{(n)}, \Theta) \right)^2$$

- **Mean absolute error (MAE) (Pérdida L1)**

$$J(\Theta) = \frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \left| y_o^{(n)} - \hat{y}_o^{(n)} \right| = \frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \left| y_o^{(n)} - f_o(\mathbf{x}^{(n)}, \Theta) \right|$$

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Regresión

- **Huber: combina MSE y MAE**

$$J(\Theta) = \frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \mathcal{L}(y_o^{(n)}, \hat{y}_o^{(n)}) = \frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \mathcal{L}(y_o^{(n)}, f_o(\mathbf{x}^{(n)}, \Theta))$$

donde

$$\mathcal{L}(y_o^{(n)}, \hat{y}_o^{(n)}) = \begin{cases} \frac{1}{2} (y_o^{(n)} - \hat{y}_o^{(n)})^2 & \text{si } |y_o^{(n)} - \hat{y}_o^{(n)}| \leq \delta \\ \delta \left(|y_o^{(n)} - \hat{y}_o^{(n)}| - \frac{1}{2} \delta \right) & \text{en otro caso} \end{cases}$$

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Regresión

Mean squared error (MSE)

- No negativa ($MSE \geq 0$): menores valores indican mejor ajuste del modelo
- Muy sensible a outliers: los errores grandes dominan la pérdida
- Continua, diferenciable y convexa (en modelos lineales): facilita la optimización por gradiente
- Dependiente de la escala: su magnitud depende de las unidades/escala de la variable objetivo
- Estándar en regresión
- Más fácil de minimizar que RMSE

Redes Perceptrón Multicapa **Funciones de Pérdida comunes** **Problemas de Regresión**

Mean absolute error (MAE)

- No negativa ($MAE \geq 0$): menores valores indican mejor ajuste del modelo.
- Menos sensible a outliers: penaliza linealmente
- Convexa (tiene mínimo global), pero no diferenciable en 0: en la práctica se usan subgradientes.
- Dependiente de la escala: para comparar entre problemas se usa Mean Absolute Percentage Error (MAPE) o Normalized MAE
- Altamente interpretable: se expresa en las mismas unidades que la variable objetivo

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Regresión

Huber

- Menos sensible a outliers que MSE: penalización lineal para errores grandes
- Convexa y diferenciable en todas partes (pero no tiene segunda derivada en $|y - \hat{y}| = \delta$)
- Elección de δ : mediante validación cruzada.
 - δ pequeño $\rightarrow$ comportamiento como MSE
 - δ grande $\rightarrow$ comportamiento como MAE

Redes Perceptrón Multicapa **Funciones de Pérdida comunes** **Problemas de Regresión**

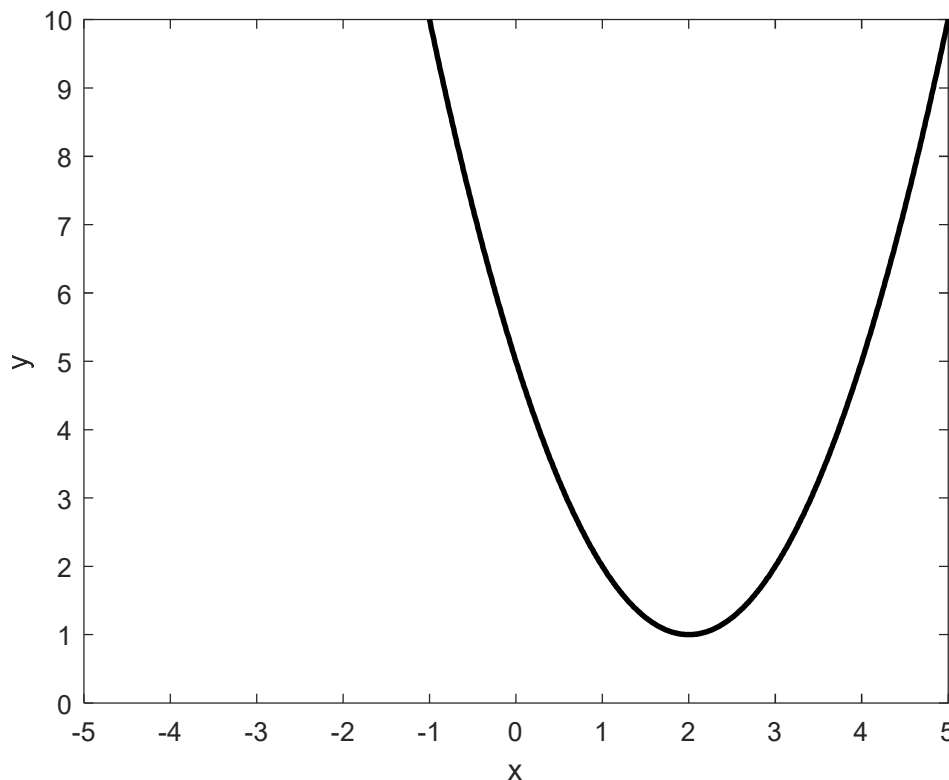
Comparación MSE-MAE-Huber

- MSE: alta sensibilidad a outliers, diferenciable, convergencia rápida cerca del mínimo
- MAE: robusta a outliers, no diferenciable en 0, gradiente constante
- Huber: balance configurable, diferenciable en todo el dominio

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Mínimo de una función $y = f(x)$



$$y = f(x) = (x - 2)^2 + 1$$

Observe que el mínimo de $f(x)$ está en el punto (2,1)

Para hallar este mínimo

$$\frac{dy}{dx} = \frac{d}{dx} f(x) = 2(x - 2) = 0$$

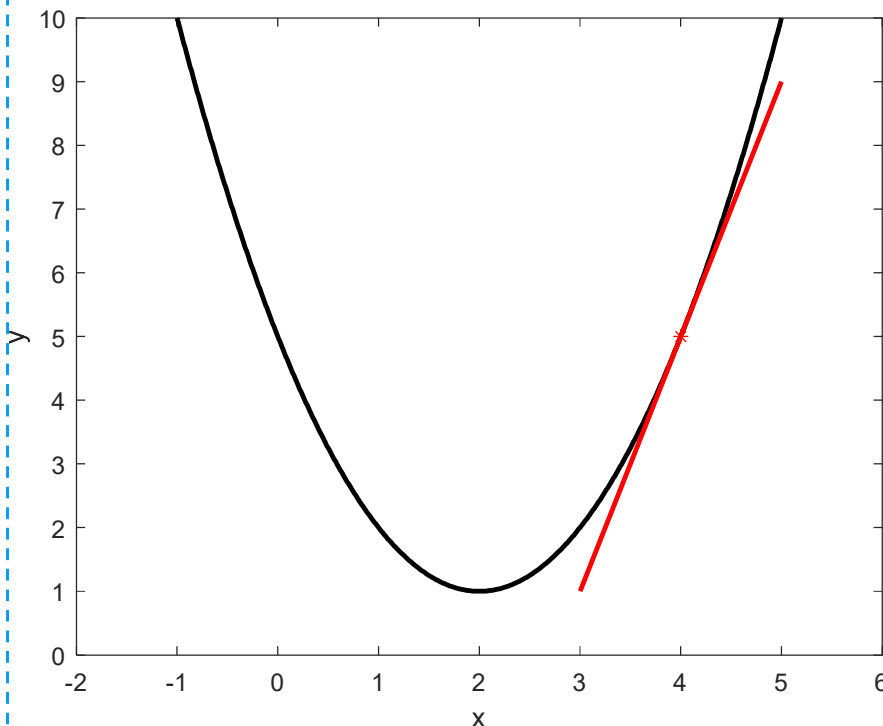
de donde $x = 2$ e

$$y = (2 - 2)^2 + 1 = 1$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Mínimo de una función $y = f(x)$



También podemos hallar el mínimo por el método (iterativo) del descenso más rápido

$$x_{i+1} = x_i - \eta \left. \frac{d}{dx} f(x) \right|_{x_i}$$

$$x_{i+1} = x_i - \eta \cdot 2(x_i - 2)$$

partiendo desde un x_0 inicial

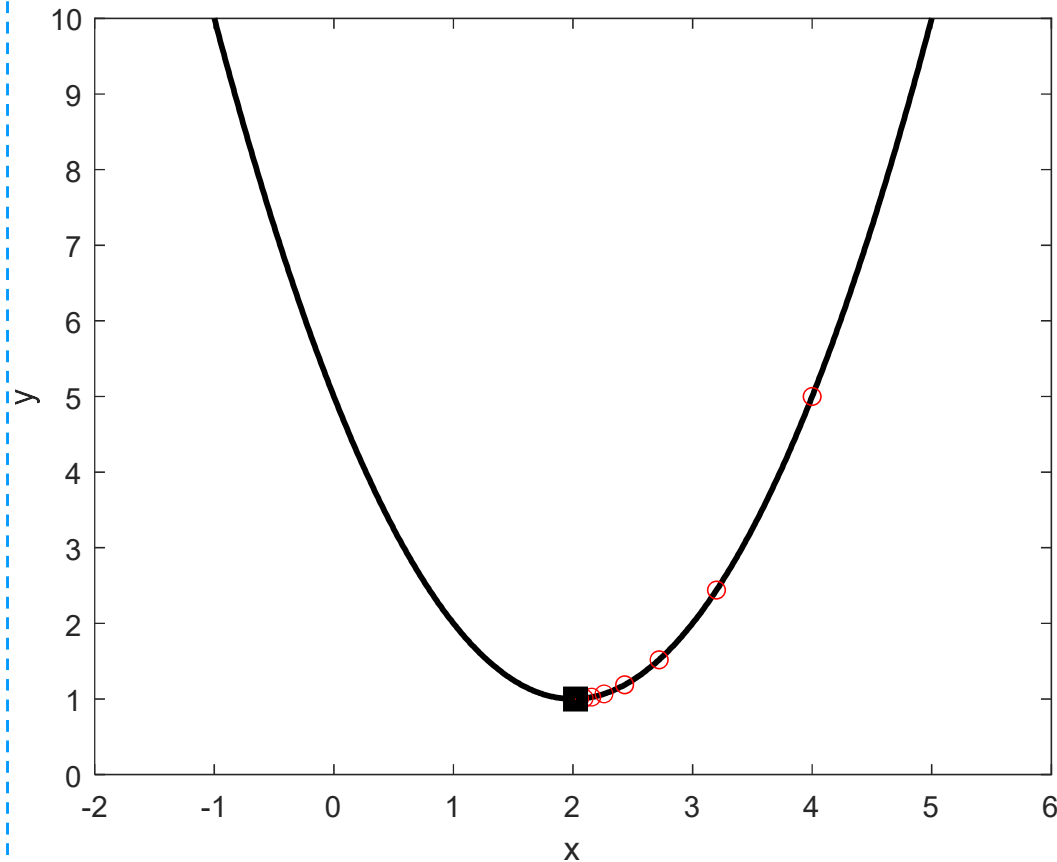
En $x_0 = 4$

$$\left. \frac{d}{dx} f(x) \right|_{x_0} = 2(4 - 2) = 4$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Mínimo de una función $y = f(x)$



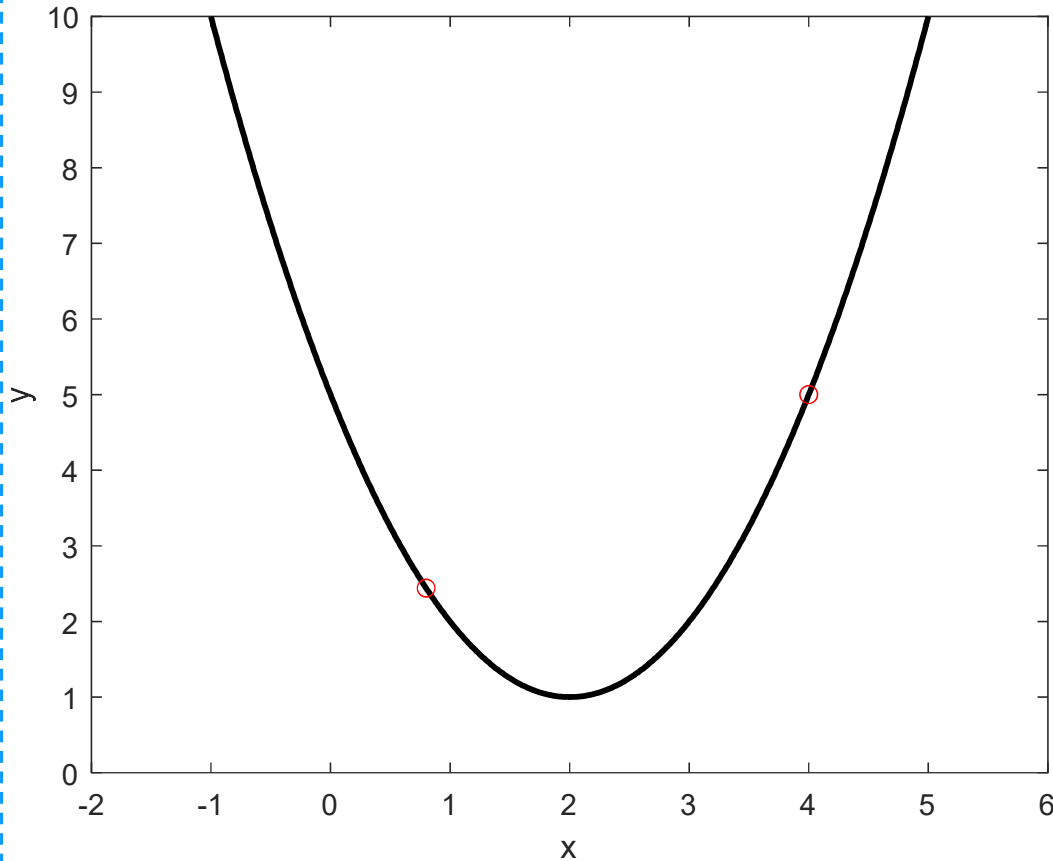
10 iteraciones con $\eta = 0.2$

i	x_i
0	4.0000
1	3.2000
2	2.7200
3	2.4320
4	2.2592
5	2.1555
6	2.0933
7	2.0560
8	2.0336
9	2.0202
10	2.0121

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Mínimo de una función $y = f(x)$



1 iteración con $\eta = 0.8$

i	x_i
0	4.0
1	0.8

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

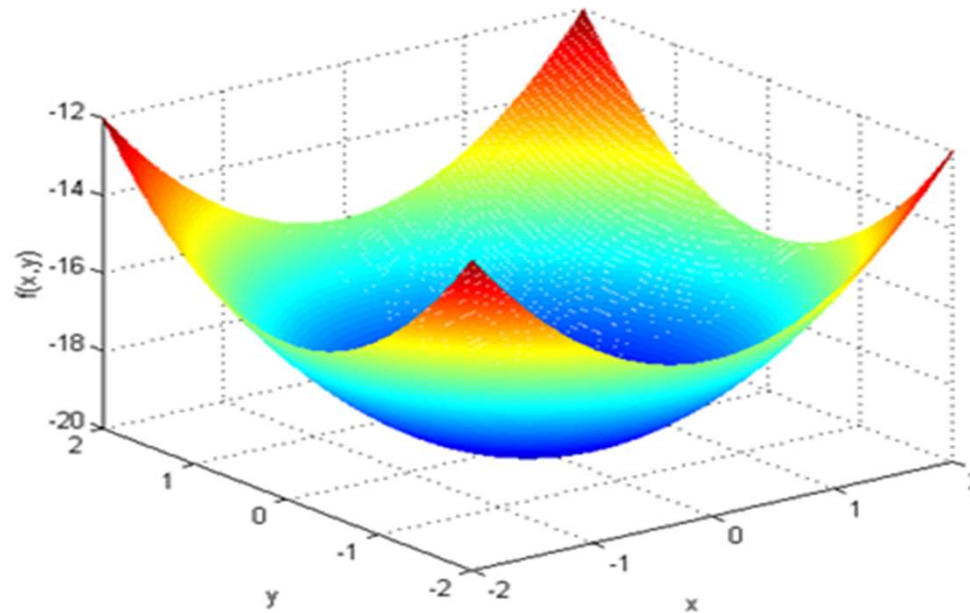
Unidad II

Método del descenso más rápido (Gradient descent)

EJEMPLO. Encuentre el mínimo de la función

$$f(x, y) = -20 + x^2 + y^2$$

Superficie de $f(x, y)$ en la región
 $\{(x, y) / x \in [-2, 2] \wedge y \in [-2, 2]\}$



Método del descenso más rápido (Gradient descent)

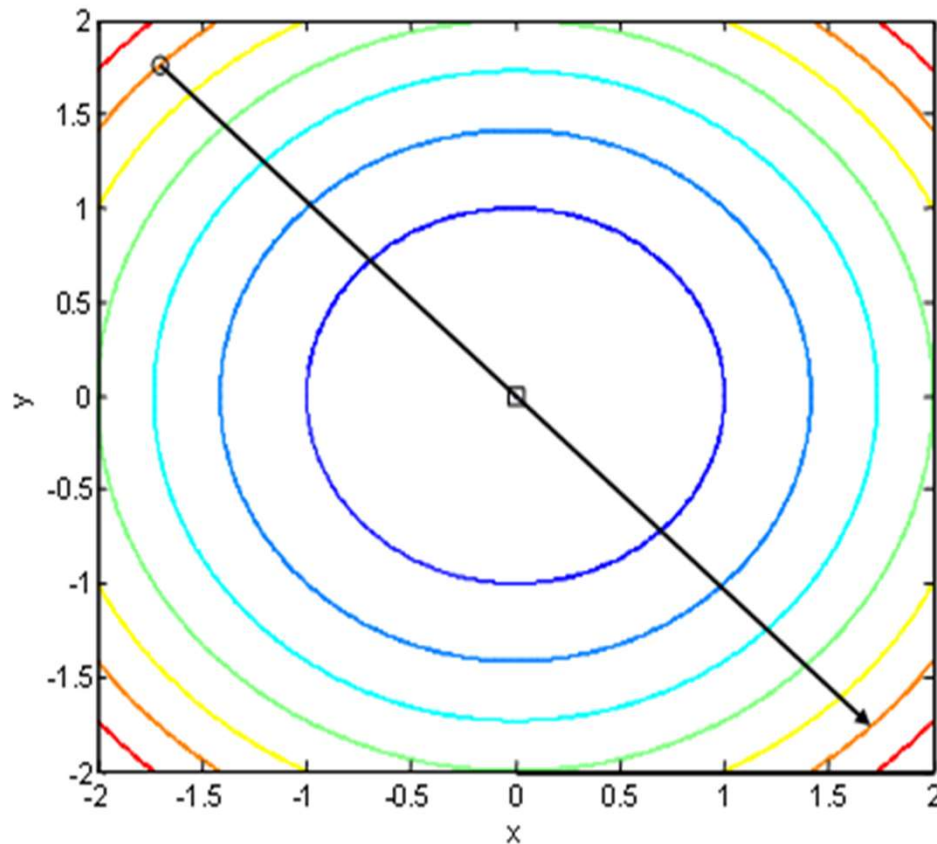
- Observe que la función tiene un único mínimo (unimodal) ubicado en $(0,0)$.
- El punto inicial se selecciona igual a $(-1.7, 1.7635)$, η es constante igual a 0.4 y $L = 20$.
- El gradiente de esta función en un punto cualquiera (x, y) es

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) = (2x, 2y)$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Método del descenso más rápido (Gradient descent)



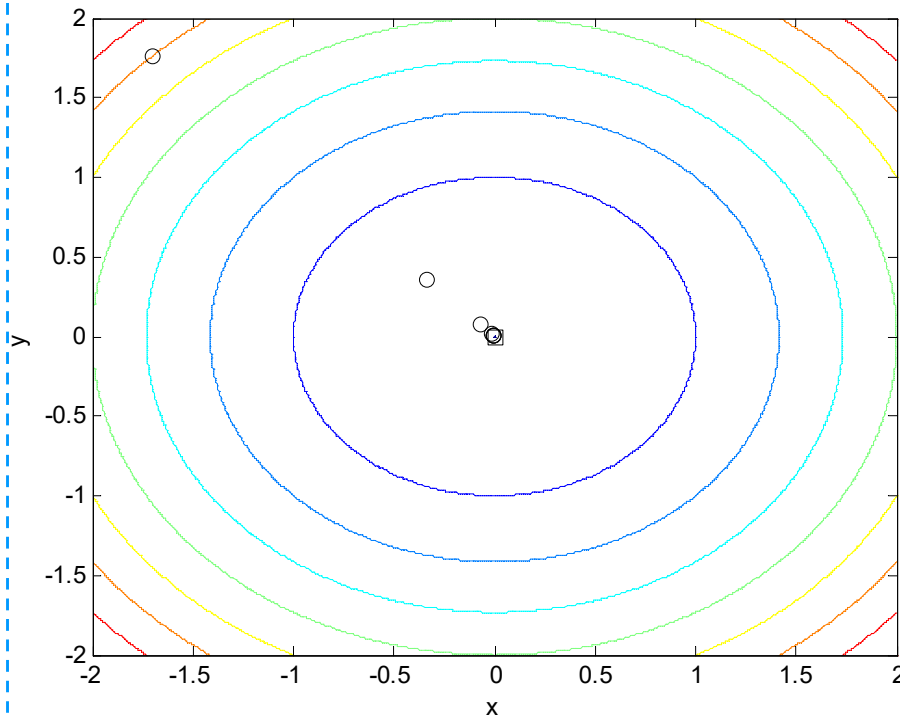
- Las curvas de nivel corresponden a círculos concéntricos con centro en el origen.
- El vector negativo del gradiente apunta hacia el mínimo.

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Método del descenso más rápido (Gradient descent)

Evolución de gradient descent con $\eta = 0.4$



$$x(k+1) = x(k) - \eta \left. \frac{\partial f(x, y)}{\partial x} \right|_{\substack{x(k) \\ y(k)}} \\ = x(k) - \eta[2x(k)]$$

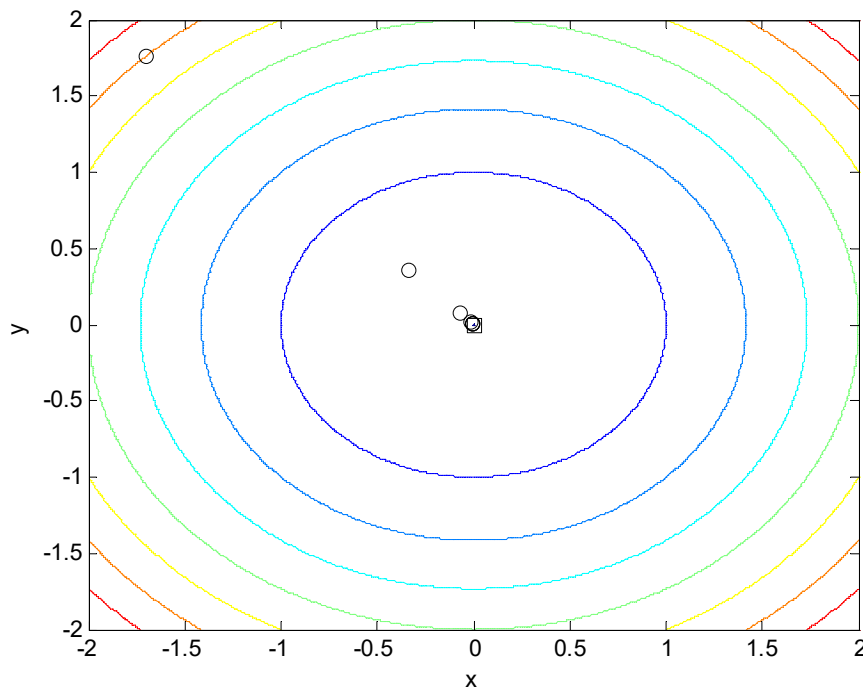
$$y(k+1) = y(k) - \eta \left. \frac{\partial f(x, y)}{\partial y} \right|_{\substack{x(k) \\ y(k)}} \\ = y(k) - \eta[2y(k)]$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Método del descenso más rápido (Gradient descent)

Evolución de gradient descent con $\eta = 0.4$



k	$x(k)$	$y(k)$
0	-1.7	1.7635
1	-0.34	0.3527
2	-0.068	0.0705
3	-0.0136	0.0141
4	-0.0027	0.0028
5	-0.0005	0.0006
6	-0.0001	0.0001

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation

- Desarrollado por Rumelhart, Hinton y Williams: publicaron el paper seminal "*Learning representations by back-propagating errors*" en Nature (1986), que popularizó el algoritmo y demostró su utilidad para entrenar redes multicapa
- Anteriormente, Paul Werbos (1974) describió backprop en su tesis doctoral en Harvard, aplicándola a redes neuronales. Fue el primero en hacerlo claramente, pero el trabajo pasó desapercibido durante años

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation

- Consideremos una red con I entradas, una capa oculta de H unidades y O salidas
- Un dataset de N datos para entrenar esta red tiene la forma

Entrada	Salida
$x_1^{(1)}, x_2^{(1)}, \dots, x_I^{(1)}$	$y_1^{(1)}, y_2^{(1)}, \dots, y_O^{(1)}$
$x_1^{(2)}, x_2^{(2)}, \dots, x_I^{(2)}$	$y_1^{(2)}, y_2^{(2)}, \dots, y_O^{(2)}$
$\vdots$	$\vdots$
$x_1^{(N)}, x_2^{(N)}, \dots, x_I^{(N)}$	$y_1^{(N)}, y_2^{(N)}, \dots, y_O^{(N)}$

Entrenamiento de una RPMC usando Backpropagation

- El algoritmo utiliza el método del descenso más rápido para minimizar la función de pérdidas
- Por ejemplo, en el caso de un problema de regresión

$$J(\Theta) = \frac{1}{2} \sum_{n=1}^N \sum_{o=1}^O \left(y_o^{(n)} - f_o(\mathbf{x}^{(n)}, \Theta) \right)^2$$

donde Θ es un vector que contiene todos los pesos de la red

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation

- El peso que conecta la unidad h de la capa oculta y la unidad o de la capa de salida se actualiza como

$$w_{oh}(k + 1) = w_{oh}(k) - \eta \frac{\partial J}{\partial w_{oh}}; o = 1, \dots, O; h = 1, \dots, H$$

donde η es el learning rate

- Puede demostrarse que para SGD

$$\frac{\partial J}{\partial w_{oh}} = - \left(y_o^{(n)} - \hat{y}_o^{(n)} \right) \left[f'_o(\sigma) |_{\sigma=a_o} \right] h_h$$

donde $f'_o(\sigma)$ es la derivada de la función de activación de las unidades de salida y $a_o = \sum_{h=1}^H w_{oh} h_h$

Entrenamiento de una RPMC usando Backpropagation

- El peso que conecta la unidad i de entrada y la unidad h de la capa oculta se actualiza como

$$v_{hi}(k + 1) = v_{hi}(k) - \eta \frac{\partial J}{\partial v_{hi}}; h = 1, \dots, H; i = 1, \dots, I$$

donde η es el learning rate

- Puede demostrarse que para SGD

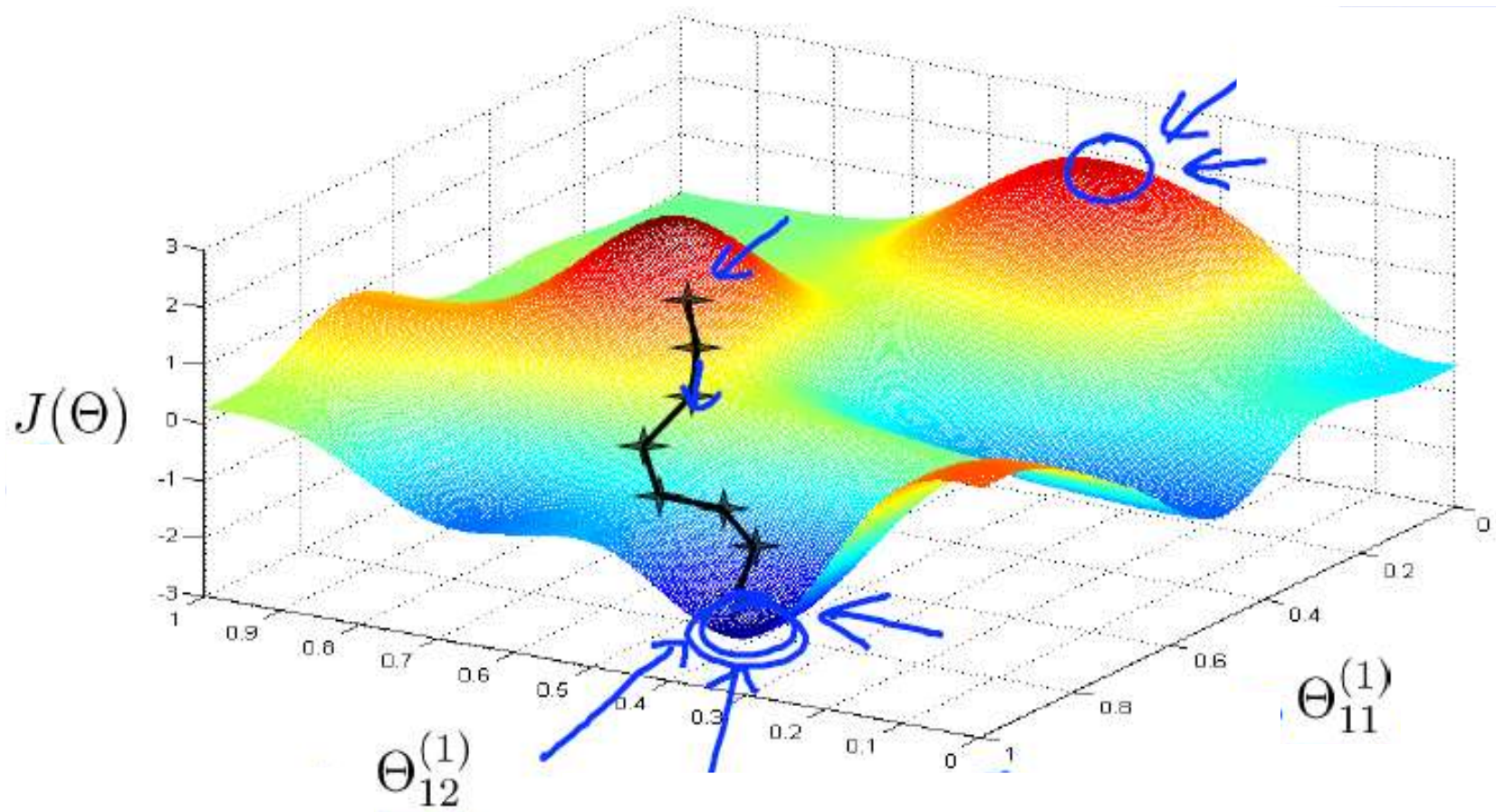
$$\frac{\partial J}{\partial v_{hi}} = - \sum_{o=1}^O \left(y_o^{(n)} - \hat{y}_o^{(n)} \right) \left[f'_o(\sigma) \Big|_{\sigma=a_o} \right] w_{oh} \left[f'_h(\sigma) \Big|_{\sigma=b_h} \right] x_i^{(n)}$$

donde $f'_h(\sigma)$ es la derivada de la función de activación de las unidades de la capa oculta y $b_h = \sum_{i=0}^I v_{hi} x_i^{(n)}$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

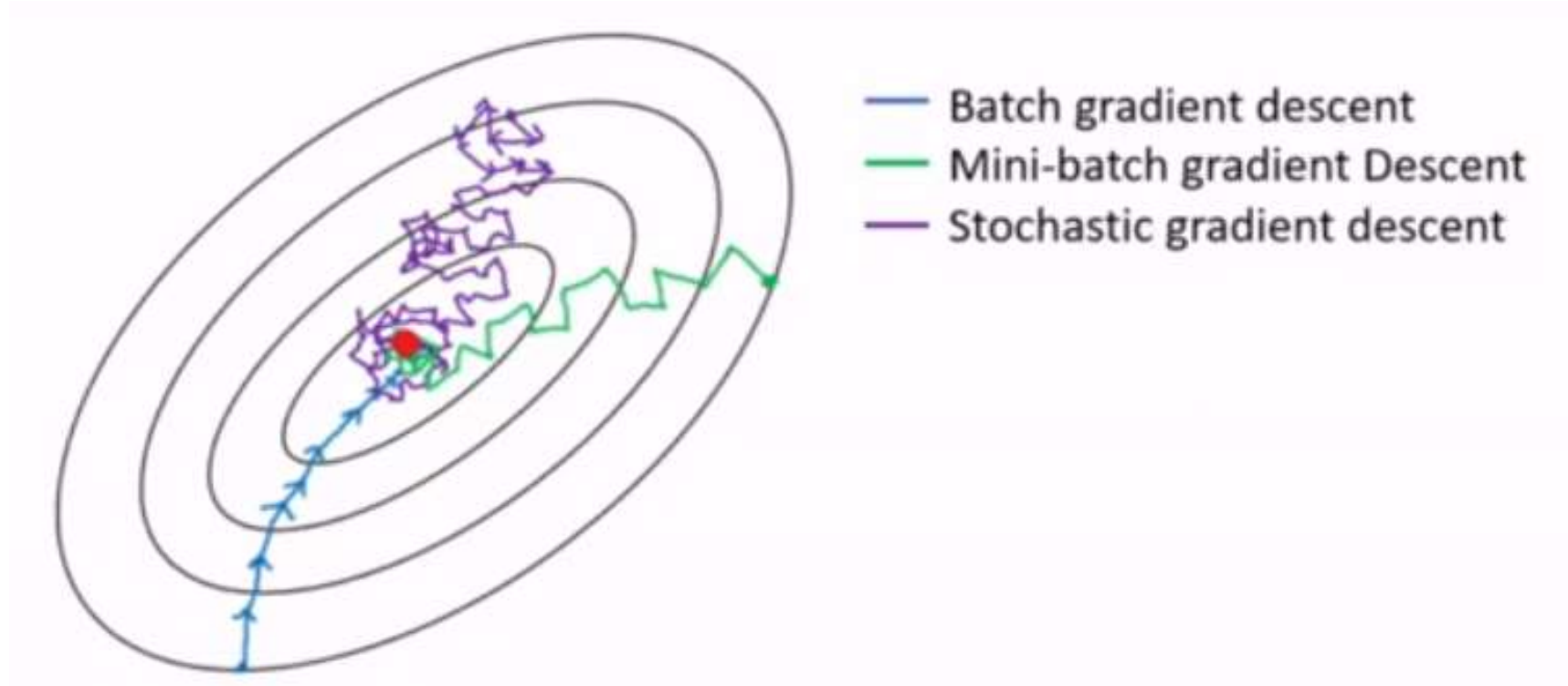
Entrenamiento de una RPMC usando Backpropagation

- Las ecuaciones de actualización de los pesos pueden implementarse de forma matricial, que es más eficiente desde el punto de vista computacional
- Los pesos pueden actualizarse luego que se presenta
 - Todos los vectores de entrenamiento (versión batch)
 - Un subconjunto de vectores de entrenamiento (versión mini-batch)
 - Un vector de entrenamiento (SGD, stochastic gradient descent)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation

- Si la red se está entrenando para resolver un problema de clasificación, se sigue un procedimiento similar para minimizar la función de pérdida
 - binary cross-entropy (problemas de clasificación binaria)
 - categorical cross-entropy (problemas multiclase)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Capacidad de Representación

¿Cuántas regiones lineales crea una red shallow?

- Con H neuronas ReLU en $1D$: hasta $H + 1$ regiones lineales
- Con H neuronas ReLU en D dimensiones: $O(H^D)$ regiones lineales

Número de parámetros vs. capacidad

- Con P parámetros totales, una red shallow tiene capacidad $\sim O(P)$ regiones
- Una red profunda con los mismos parámetros puede tener capacidad exponencialmente mayor

Red Shallow vs Red Deep

- Red shallow: H neuronas $\rightarrow O(H)$ regiones en $1D$
- Red de L capas:
 - En $1D$: H neuronas por capa $\rightarrow O(H^L)$ regiones
 - En D dimensiones: H neuronas por capa $\rightarrow O(H^{DL})$ regiones

La profundidad aumenta capacidad de forma exponencial con el número de capas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Capacidad de Representación

Conclusión clave

- El teorema de aproximación universal garantiza existencia, no eficiencia
- Representar algunas funciones con una red shallow requiere significativamente más parámetros

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Limitaciones de las Redes Shallow

Problema 1: Ineficiencia de parámetros

- Para representar funciones 'compuestas' ($f = g \circ h$), una red shallow necesita exponencialmente más neuronas que una red profunda
- Ejemplo: reconocer una cara $\rightarrow$ bordes $\rightarrow$ partes $\rightarrow$ rostro. Una sola capa no puede aprovechar esta jerarquía
- Funciones que se representan eficientemente con depth L requieren exponencialmente más unidades con depth $L - 1$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Limitaciones de las Redes Shallow

Problema 2: No captura estructura jerárquica

- El mundo real tiene estructura de composición:
 - Objetos = partes; oraciones = palabras; señales = frecuencias
- Las redes profundas aprenden representaciones jerárquicas de forma natural

Problema 3: Generalización

- Redes muy anchas (shallow con H grande) tienden a sobreajustar con datos limitados
- La inducción implícita de la profundidad actúa como regularizador

Motivación para la Profundidad

Argumento 1: Eficiencia exponencial

- Una red de L capas con H neuronas representa $O(H^L)$ regiones lineales
- Una red shallow necesita H^L neuronas para el mismo poder expresivo

Argumento 2: Composición de funciones simples

- Cada capa aprende una transformación del espacio de representación
- Capas sucesivas construyen abstracciones progresivamente más complejas
- Análogo al razonamiento humano: de lo concreto a lo abstracto

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Motivación para la Profundidad

Argumento 3: Evidencia empírica en visión

- Capa 1 → bordes y gradientes de color
- Capa 2 → texturas y esquinas
- Capa 3 → partes de objetos
- Capa 4+ → objetos completos y conceptos

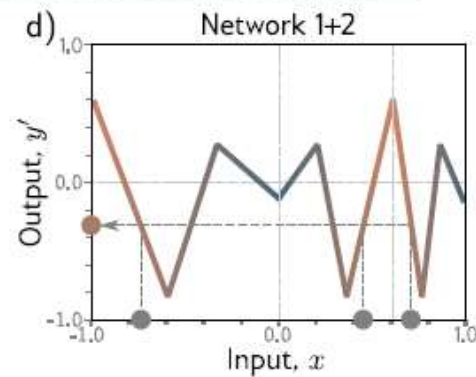
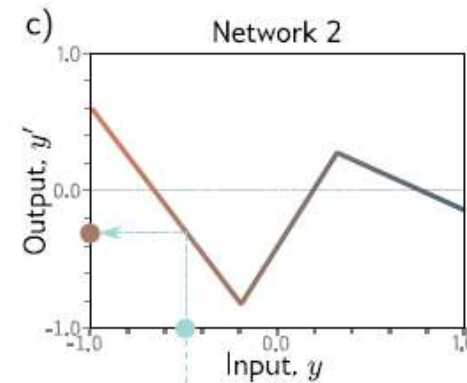
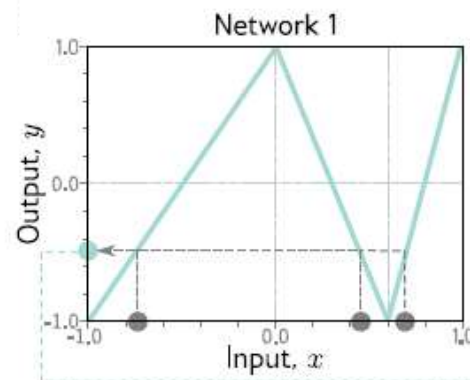
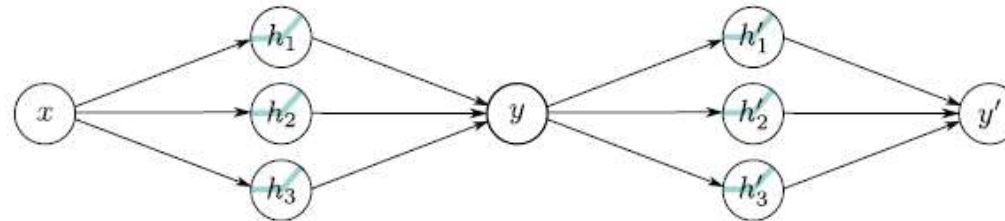
En resumen:

- 'Adding more layers to a network generally leads to better performance'
- No linealidad iterada crea fronteras de decisión arbitrariamente complejas
- La profundidad transforma representaciones no linealmente separables en linealmente separables

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

De Shallow a Deep



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Contenido

- Backpropagación vectorizada
 - Regla de la cadena y grafo computacional
 - Derivadas de pérdida respecto a cada capa
 - Algoritmo completo: paso a paso con matrices
- Implementación en PyTorch: autograd y nn.Module
- Regularización
 - L1, L2, Dropout, Batch Normalization, Layer Normalization
- Optimizadores
 - SGD con momentum, Adam, AdamW
 - Learning rate scheduling
- Inicialización de pesos: Xavier, He
- Debugging del entrenamiento
 - Gradient flow, vanishing/exploding gradients, sanity checks

Retropropagación (Backpropagation)

Observación práctica

- Una red más profunda y estrecha suele generalizar mejor que una más ancha y shallow
- El mismo número de parámetros distribuido en más capas → mejor performance

Objetivo

- Actualizar los parámetros de una red para reducir la función de pérdidas J :

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial J}{\partial \theta}$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Backpropagation

¿Cómo calcular $\partial J / \partial W$ para cada capa?

- La pérdida J depende de la salida, que depende de la última capa
- La última capa depende de la penúltima, y así sucesivamente
- Aplicación de la regla de la cadena al grafo computacional de una red neuronal para propagar la señal de error desde la salida hacia los pesos de las primeras capas
- Complejidad computacional: $O(\text{parámetros})$ — igual que un forward pass

Dos operaciones principales

- Forward pass: calcular y guardar las activaciones de cada capa
- Backward pass: propagar gradientes desde la salida hacia la entrada

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Backpropagación Vectorizada

- En la práctica trabajamos con matrices (batches de N muestras)

Notación

- $H^l \in \mathbb{R}^{N \times H_l}$ — activaciones de la capa l para el batch
- $W^l \in \mathbb{R}^{H_l \times H_{l-1}}$ — pesos de la capa l
- $\delta^l \equiv \frac{\partial J}{\partial Z^l} \in \mathbb{R}^{N \times H_l}$ — gradiente de la pérdida respecto a las preactivaciones

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Algoritmo Backpropagation Vectorizado

FORWARD PASS — guardar activaciones

Entrada (capa 0):

$$H^0 = X$$

Preactivación capa l :

$$Z^l = H^{l-1} W^{l\top} + b^l$$

Activación capa l (f es la función de activación):

$$H^l = f(Z^l)$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Algoritmo Backpropagation Vectorizado

BACKWARD PASS — propagar gradientes

Propagación del error hacia la capa anterior ($\odot$ = Hadamard):

$$\delta^l = (\delta^{l+1} W^{l+1}) \odot f'(Z^l)$$

Gradiente de la pérdida respecto a los pesos W^l :

$$\frac{\partial L}{\partial W^l} = \delta^{l\top} H^{l-1}$$

Gradiente de la pérdida respecto al bias b^l (suma sobre el batch):

$$\frac{\partial L}{\partial b^l} = \sum_n \delta_n^l$$

Gradiente respecto a las activaciones de la capa anterior:

$$\frac{\partial L}{\partial H^{l-1}} = \delta^l W^l$$

Aplicar derivada de la activación para obtener δ^l :

$$\delta^l = \left(\frac{\partial L}{\partial H^l} \right) \odot f'(Z^l)$$

Algoritmo Backpropagation Vectorizado **ACTUALIZACIÓN (Gradient Descent)**

Actualización de pesos:

$$W^l \leftarrow W^l - \eta \frac{\partial L}{\partial W^l}$$

Actualización de bias:

$$b^l \leftarrow b^l - \eta \frac{\partial L}{\partial b^l}$$

Implementación de un MLP en PyTorch

nn.Module — La Base de PyTorch DL

- nn.Module es la clase base para todos los modelos en PyTorch

Responsabilidades de nn.Module

- Contiene los parámetros del modelo (nn.Parameter)
- Define el grafo computacional en forward()
- Gestiona sub-módulos jerárquicamente (.children(), .modules())
- Permite mover el modelo entre dispositivos (.to(device), .cuda())
- Alternancia entre modos entrenamiento y evaluación (.train(), .eval())
- Serialización: guardar y cargar pesos (.state_dict(), .load_state_dict())

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Implementación de un MLP en PyTorch

nn.Module — La Base de PyTorch DL

Capas predefinidas más comunes

- `nn.Linear(in, out)`: capa completamente conectada
- `nn.ReLU()`, `nn.GELU()`, `nn.SiLU()`: funciones de activación
- `nn.BatchNorm1d(features)`: batch normalization
- `nn.Dropout(p)`: dropout
- `nn.Sequential`: apila capas en orden secuencial

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Regularización — Motivación

- Las redes neuronales profundas tienen millones de parámetros: alto riesgo de overfitting

Trade-off Bias-Varianza en DL

- Underfitting (alto bias): modelo demasiado simple, error alto en train y test
- Overfitting (alta varianza): memoriza el training set, falla en test

Señales de overfitting

- Brecha grande entre train loss y validation loss
- Validation loss sube mientras train loss sigue bajando
- Alta accuracy en train, baja en validation

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Regularización — Motivación

Estrategias de regularización

- Restricción de parámetros: L1/L2 weight decay
- Ruido durante entrenamiento: Dropout
- Normalización de activaciones: Batch Norm, Layer Norm
- Data augmentation
- Early stopping: parar cuando val_loss deja de mejorar

Regularización L2 — Weight Decay

Función de pérdida regularizada con L2 (norma de Frobenius)

$$L_{reg} = L + \frac{\lambda}{2} \sum_l \|W^l\|^2$$

donde λ es el coeficiente de regularización (hiperparámetro)

Gradiente

$$\frac{\partial L_{reg}}{\partial W^l} = \frac{\partial L}{\partial W^l} + \lambda W^l$$

Actualización — factor $(1 - \eta\lambda)$ produce el efecto "weight decay":

$$W^l \leftarrow (1 - \eta\lambda)W^l - \eta \frac{\partial L}{\partial W^l}$$

Interpretación

- Penaliza pesos grandes: los fuerza a mantenerse pequeños
- Promueve soluciones suaves y generalizables

Regularización L1 — Sparsity

Función de pérdida regularizada con L1

$$L_{reg} = L + \lambda \sum_{i,j} |w_{ij}|$$

Gradiente L1 — penalización constante (no depende del tamaño del peso):

$$\frac{\partial L_{reg}}{\partial w_{ij}} = \frac{\partial L}{\partial w_{ij}} + \lambda \cdot \text{sign}(w_{ij})$$

Propiedad de sparsity

- L1 empuja los pesos exactamente a cero → soluciones sparse
- L2 reduce los pesos pero rara vez los lleva a exactamente cero

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa L1 vs L2

- L1: sparse, selección implícita de features, menos estable con SGD
- L2: pesos pequeños pero no nulos, más estable, equivalente a weight decay

Uso en DL

- L2 (weight decay) es estándar en casi todos los modelos modernos
- L1 se usa menos en DL profundo — más común en regresión lineal/logística (Lasso)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Dropout (Srivastava et al., 2014)

Mecanismo

- Durante entrenamiento: cada neurona se desactiva independientemente con probabilidad p
- Las neuronas activas se escalan por $1/(1-p)$ para mantener la esperanza
- Durante evaluación: todas las neuronas activas — no hay dropout

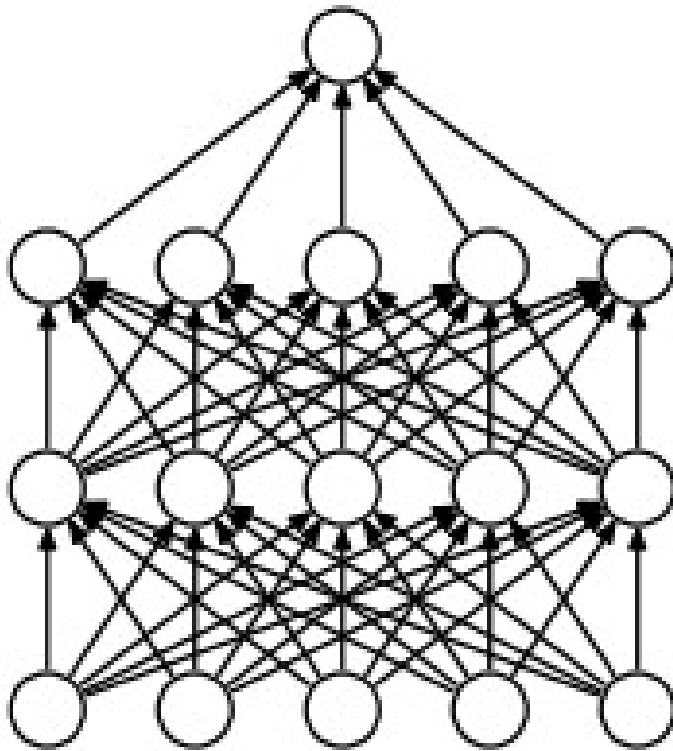
Implementación

- `model.train()`: activa el dropout
- `model.eval()`: desactiva el dropout — (No olvidarlo)

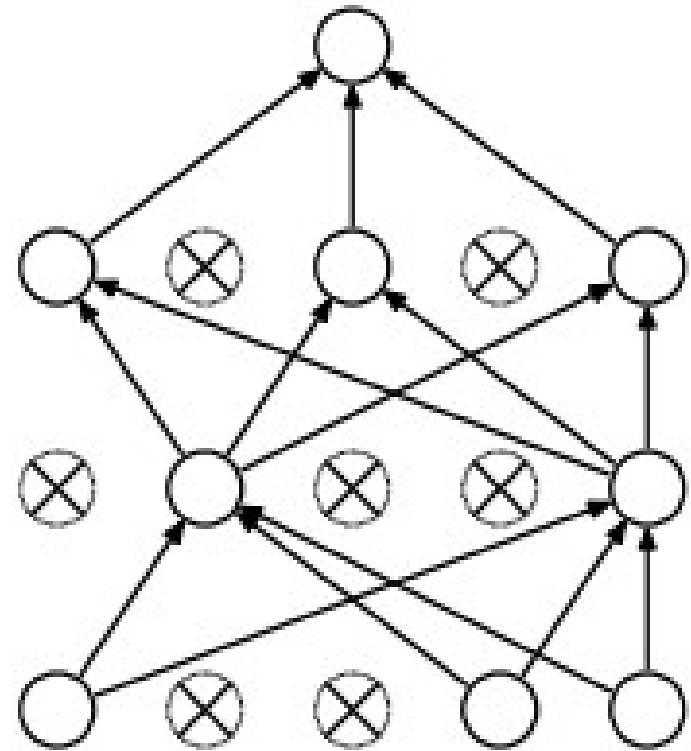
INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Dropout (Srivastava et al., 2014)



Red neuronal estándar



Red con dropout aplicado

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Dropout

¿Por qué funciona? — Interpretaciones

- Ensemble: entrena simultáneamente 2^n sub-redes — evaluación es el promedio implícito
- Ruido: añade ruido aleatorio que previene la co-adaptación de neuronas
- Robustez: la red aprende a no depender de ninguna neurona específica

Tasas típicas de dropout

- Capas densas: $p = 0.3 - 0.5$
- Capas de entrada: $p = 0.1 - 0.2$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Batch Normalization (Ioffe & Szegedy, 2015)

Problema que resuelve: Internal Covariate Shift

- La distribución de activaciones de cada capa cambia durante el entrenamiento
- Las capas posteriores deben adaptarse constantemente a distribuciones variables
- Esto frena el entrenamiento y requiere learning rates pequeños

Beneficios

- Permite learning rates más grandes → entrenamiento más rápido
- Regularización implícita → menos necesidad de dropout
- Reduce sensibilidad a la inicialización

¿Antes o después de la activación?

- Original (Ioffe & Szegedy): BN antes de la activación
- Pre-Norm (práctica moderna): BN antes de cada bloque, activación después

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Batch Normalization (Ioffe & Szegedy, 2015)

Algoritmo — por batch y por feature

Media del mini-batch (N muestras):

$$\mu_B = \frac{1}{N} \sum_n z_n$$

Varianza del mini-batch:

$$\sigma_B^2 = \frac{1}{N} \sum_n (z_n - \mu_B)^2$$

Normalización (ϵ para estabilidad numérica):

$$\widehat{z}_n = \frac{z_n - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Escalado y desplazamiento (parámetros aprendibles γ , β):

$$y_n = \gamma \cdot \widehat{z}_n + \beta$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Batch Normalization — Detalles Importantes

Durante entrenamiento

- μ y σ^2 se calculan sobre el mini-batch actual
- Se actualiza un running mean y running var (media exponencial móvil)

Durante evaluación (`model.eval()`)

- Se usan los running mean y running var acumulados durante el entrenamiento
- Si se olvida `model.eval()`: BN usa stats del batch de test → resultados incorrectos

Parámetros aprendibles (γ , β)

- γ (scale) y β (shift): permiten que la red revierta la normalización si es necesario
- Inicialmente: $\gamma = 1$, $\beta = 0$

Batch Normalization — Detalles Importantes

Limitación principal

- Depende del tamaño del batch: batch pequeño → estimaciones de μ, σ^2 ruidosas
- Con `batch_size=1`: totalmente inestable
- Alternativa: Layer Normalization (independiente del batch)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Layer Normalization (Ba et al., 2016)

Diferencia clave con Batch Norm

- BN normaliza a través del batch ($\text{dim}=0$): calcula μ, σ sobre las N muestras
- LN normaliza a través de los features ($\text{dim}=-1$): calcula μ, σ para cada muestra

Ventajas sobre Batch Norm

- Independiente del tamaño del batch — funciona con `batch_size=1`
- Idéntico en training y evaluation — sin modo dual
- Estándar en todos los Transformers (BERT, GPT, ViT)

¿Cuándo usar cuál?

- Batch Norm: CNNs, MLPs con batch grande — práctica común en visión
- Layer Norm: Transformers, RNNs, NLP — cualquier arquitectura con batch variable

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Layer Normalization (Ba et al., 2016)

Algoritmo

Media por muestra (sobre los H features):

$$\mu_n = \frac{1}{H} \sum_h z_{nh}$$

Varianza por muestra:

$$\sigma_n^2 = \frac{1}{H} \sum_h (z_{nh} - \mu_n)^2$$

Normalización:

$$\widehat{z}_{nh} = \frac{z_{nh} - \mu_n}{\sqrt{\sigma_n^2 + \varepsilon}}$$

Escalado y desplazamiento (γ_h, β_h son parámetros por feature):

$$y_{nh} = \gamma_h \cdot \widehat{z}_{nh} + \beta_h$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Gradient Descent — Revisión y Limitaciones

SGD básico

- $\theta_{t+1} = \theta_t - \eta \cdot g_t$ donde $g_t = \partial L / \partial \theta$
- Problema: muy sensible al learning rate
- Problema: misma LR para todos los parámetros — ineficiente
- Problema: oscila en valles alargados (plateaus y ravines)

Problemas

- Saddle points: más comunes que mínimos locales en alta dimensión
- Ravines: curvatura alta en una dirección, baja en otra → zigzag lento
- Plateaus: gradientes casi nulos durante largas regiones

Soluciones

- Momentum: acumular historia de gradientes para suavizar trayectoria
- Adaptive learning rates: ajustar η por parámetro según su historial

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

SGD con Momentum

Algoritmo

Velocidad — media exponencial móvil del gradiente:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t$$

Actualización con momentum:

$$\theta_{t+1} = \theta_t - \eta \cdot v_t$$

donde β es el coeficiente momentum

Intuición física

- La pelota acumula velocidad mientras rueda cuesta abajo
- En direcciones consistentes: velocidad aumenta, convergencia más rápida
- En direcciones oscilantes: la velocidad se cancela, reducción del zigzag

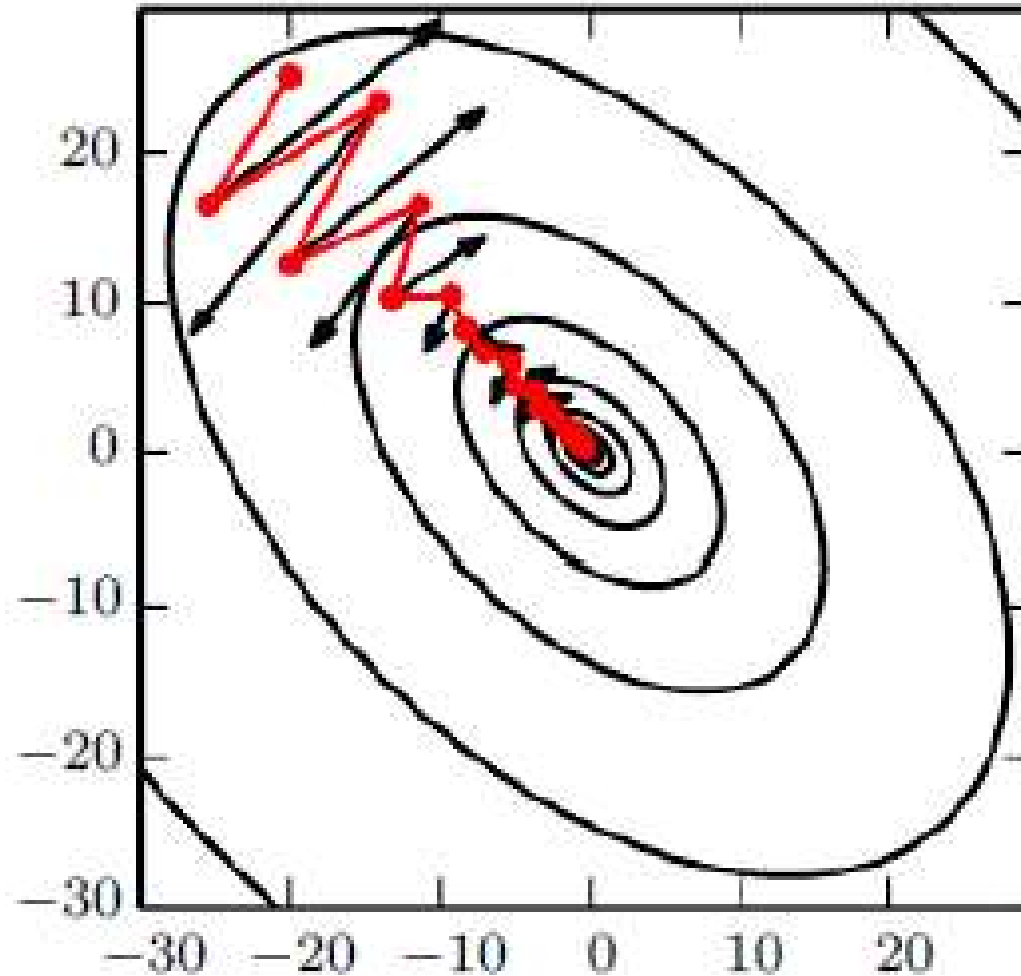
Valores típicos

- $\eta = 0.01 - 0.1$ (mucho mayor que sin momentum)
- $\beta = 0.9$ — valor estándar en casi todos los trabajos

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

SGD con Momentum



RMSprop — Adaptive Learning Rates

RMSprop (Hinton, 2012)

Media exponencial del cuadrado del gradiente ($\rho \approx 0.9$):

$$s_t = \rho \cdot s_{t-1} + (1 - \rho) \cdot g_t^2$$

Actualización con learning rate adaptativa:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t} + \varepsilon} \cdot g_t$$

Efecto: LR grande para parámetros con gradientes pequeños y viceversa

Cuándo usar RMSprop

- Útil para RNNs (gradientes muy variables en el tiempo)
- En la práctica: Adam (que combina momentum + RMSprop) es más popular

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Adam — Adaptive Moment Estimation

Kingma & Ba (2014) — el optimizador más usado en DL

- Combina momentum (primer momento) con RMSprop (segundo momento)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Adam — Adaptive Moment Estimation

Algoritmo

Primer momento — media exponencial de gradientes ($\beta_1 = 0.9$):

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

Segundo momento — media del cuadrado del gradiente ($\beta_2 = 0.999$):

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

Corrección de bias — primer momento (importante al inicio del entrenamiento):

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

Corrección de bias — segundo momento:

$$\widehat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Actualización Adam ($\eta = 10^{-3}$ por defecto, $\varepsilon = 10^{-8}$):

$$\theta_{t+1} = \theta_t - \frac{\eta \cdot \widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Adam — Adaptive Moment Estimation

Hiperparámetros por defecto (rara vez se cambian)

- $\beta_1 = 0.9$ (momentum), $\beta_2 = 0.999$ (escala), $\varepsilon = 10^{-8}$
- $\eta = 10^{-3}$ — valor estándar de partida

Intuición

- $\widehat{m}_t$: dirección suavizada del descenso
- $\sqrt{\widehat{v}_t}$: escala adaptativa — parámetros con gradientes grandes tienen LR efectiva menor

AdamW — Adam con Weight Decay Correcto

El problema con Adam + L2

- En Adam, añadir $\lambda ||W||^2$ a la pérdida e incluirlo en g_t no equivale a weight decay puro
- El L2 en Adam queda amortiguado por $\sqrt{\widehat{v}_t} \rightarrow$ diferentes parámetros reciben decay diferente

AdamW (Loshchilov & Hutter, 2019) — la corrección

- Separar el weight decay de la actualización de Adam:
- Actualización AdamW — el weight decay actúa directamente sobre θ :

$$\theta_{t+1} = \theta_t - \frac{\eta \cdot \widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon} - \eta \cdot \lambda \cdot \theta_t$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

AdamW — Adam con Weight Decay Correcto

Impacto práctico

- AdamW generaliza significativamente mejor que Adam con L2 en la pérdida
- Adoptado como estándar en: GPT, BERT, ViT, ResNet modernos, YOLO

Guía de uso

- Empezar siempre con AdamW — es raro que SGD o Adam puro supere a AdamW
- `weight_decay`: 0.01-0.1 para modelos grandes; $1e-4$ para modelos pequeños

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa de Optimizadores

Optimizador	LR adaptativa	Momentum	Weight Decay	Uso típico
SGD	No	Opcional	Vía L2	CNNs clásicas, ImageNet
SGD+Nesterov	No	Sí	Vía L2	CV con batch grande
RMSprop	Sí	No	Vía L2	RNNs, RL
Adam	Sí	Sí	Problemático	NLP, atención, prototipos
AdamW	Sí	Sí	✓ Correcto	Estándar moderno (GPT, ViT, YOLO)

Recomendación práctica

- AdamW como punto de partida en prácticamente todos los proyectos
- SGD+momentum para CNNs en visión con LR scheduling agresivo (OneCycle, Cosine)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Learning Rate Scheduling

¿Por qué no usar un LR fijo?

- LR grande al inicio: converge rápido pero puede oscilar cerca del mínimo
- LR pequeño al inicio: muy lento, puede quedarse atrapado en plateaus
- Solución: LR grande al inicio, reducir progresivamente al acercarse al mínimo

Learning Rate Scheduling

Estrategias principales

- Step decay: reducir LR por factor γ cada N épocas
- Cosine Annealing: LR sigue un coseno de η_{max} a η_{min}
- OneCycleLR: aumentar hasta η_{max} y luego bajar (Smith, 2019)
- Warmup lineal: LR crece linealmente de 0 a η_{max} en los primeros K steps
- ReduceLROnPlateau: reduce LR cuando la val_loss no mejora

Warmup + Cosine Decay (estándar en Transformers y ViT)

- Fase 1 (warmup): LR crece linealmente de 0 a η_{max} en T_{warmup} steps
- Fase 2: LR sigue coseno decreciente hasta η_{min}
- Evita inestabilidad al inicio cuando los pesos aún son aleatorios

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

¿Por qué Importa la Inicialización?

Dos extremos problemáticos

- Pesos demasiado grandes: activaciones explotan → saturación o gradientes explosivos
- Pesos demasiado pequeños: activaciones se desvanecen → vanishing gradients

Mala inicialización → fracaso del entrenamiento

- Redes de 10+ capas con pesos aleatorios $N(0,1)$: training imposible sin BN
- Con buena inicialización: el entrenamiento puede comenzar de forma estable

El objetivo

- La varianza de las activaciones debe mantenerse aproximadamente constante capa a capa
- La varianza de los gradientes también debe mantenerse al propagar hacia atrás

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Inicialización He / Kaiming

He et al. (2015) — 'Delving deep into rectifiers'

- Diseñada específicamente para ReLU y variantes

Algoritmo

Varianza He — compensa el factor $\frac{1}{2}$ de ReLU ($\approx 50\%$ de neuronas desactivadas):

$$Var(w) = \frac{2}{D_{in}}$$

Distribución uniforme He:

$$w \sim Uniform\left(-\sqrt{\frac{6}{D_{in}}}, +\sqrt{\frac{6}{D_{in}}}\right)$$

Distribución normal He:

$$w \sim \mathcal{N}\left(0, \sqrt{\frac{2}{D_{in}}}\right)$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Gradient Flow — Diagnóstico

- Monitorear el flujo de gradientes es crítico para detectar problemas de entrenamiento

¿Qué monitorear?

- Norma del gradiente por capa: $||\partial L / \partial W^1||$
- Histograma de gradientes: distribución por capa y por época
- Ratio gradiente/peso: $||\partial L / \partial W^1|| / ||W^1||$ — debe ser $\sim 10^{-3}$

Gradient Flow — Diagnóstico

Señales de vanishing gradient

- Las primeras capas tienen gradientes órdenes de magnitud menores que las últimas
- La pérdida no disminuye aunque el modelo tenga capacidad suficiente
- Los pesos de las primeras capas no cambian entre épocas

Señales de exploding gradient

- La pérdida diverge (NaN o Inf)
- Los pesos crecen sin control entre épocas
- Solución: gradient clipping

$$g_t \leftarrow g_t \cdot \min\left(1, \frac{\text{max_norm}}{\|g_t\|}\right)$$

Herramienta

- W&B: gradient histograms en tiempo real durante el entrenamiento

Sanity Checks — Antes de Entrenar

- Antes de lanzar un entrenamiento largo, verificar que todo funciona correctamente

Check 1: pérdida inicial correcta

- Con softmax: pérdida inicial esperada $\approx \log(K)$ para K clases
- Con $K=10$ (MNIST): $\log(10) \approx 2.30$ — si la pérdida inicial es muy distinta, hay un bug

Check 2: overfit en un batch pequeño

- Entrenar solo en 5-20 muestras durante muchas épocas
- El modelo debe lograr training loss ≈ 0 — si no, hay un bug en el modelo o en la pérdida

Sanity Checks — Antes de Entrenar

Check 3: verificar shapes

- Imprimir shapes de X, y, logits, loss en el primer batch
- Verificar que el forward pass produce la shape esperada

Check 4: gradient check numérico

- `torch.autograd.gradcheck()` — compara gradientes analíticos con diferencias finitas
- Útil cuando se implementan capas propias

Check 5: monitorear la primera época

- La pérdida debe bajar monotónicamente en el primer batch si el LR es correcto

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Diagnóstico — Problemas Comunes

La pérdida no baja

- LR demasiado pequeño → aumentar 10x y ver si hay movimiento
- Gradientes cerca de 0 → revisar funciones de activación e inicialización
- Bug en la pérdida → sanity check con batch pequeño

La pérdida diverge o sale NaN

- LR demasiado grande → reducir 10x
- Gradientes exploding → aplicar gradient clipping (max_norm=1.0)
- Overflow en $\exp()$ → usar log-sum-exp trick, revisar softmax con LogSoftmax

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Diagnóstico — Problemas Comunes

Overfitting

- Aumentar dropout, añadir weight decay, reducir capacidad del modelo
- Data augmentation, más datos

Underfitting

- Aumentar capacidad: más capas, más neuronas
- Reducir regularización, entrenar más épocas
- Revisar si hay bug en el preprocesamiento de datos

Entrenamiento lento

- Verificar que los datos están en GPU, no solo el modelo
- Aumentar batch_size si la VRAM lo permite